LoRA: Towards Improved Applicability of Reconfigurable Architecture for Versatile Nonlinear Functions

Yuan Dai*, Guibin Zou*, Yuanda Yang, Huan Lin, Jiahang Lou, Yiwen Luo, Xinyu Cai,
Wenbo Yin, Wai-Shing Luk, Lingli Wang✉
State Key Laboratory of Integrated Chips and Systems, Fudan University, China
Email: daiy21@m.fudan.edu.cn, llwang@fudan.edu.cn

2026 ACM/IEEE 53rd Annual International Symposium on Computer Architecture (ISCA) | 979-8-3315-5065-3/26/$31.00 ©2026 IEEE | DOI: 10.1109/ISCA66397.2026.00068

***Abstract*—Coarse-Grained Reconfigurable Architecture (CGRA) emerges as a competitive accelerator for computation-intensive applications. However, most CGRAs primarily focus on linear operations, which makes it challenging to efficiently support nonlinear functions that are increasingly important in emerging applications. This limitation hinders the potential of CGRAs to accelerate modern computation-intensive applications, particularly those that rely on complex nonlinear functions.**

In this paper, we propose several key approaches to enable CGRA to support versatile nonlinear functions. First, we develop a Chebyshev-based approximation algorithm that employs a novel segmentation strategy, along with considering the properties of the target function to identify a suitable Chebyshev polynomial for each sub-interval. Second, based on the Logarithmic Number System, we design a lightweight unit to efficiently compute the polynomials and complex operations. Finally, we develop a comprehensive end-to-end framework *LoRA* with architectural and compiler support for implementing nonlinear functions on the CGRA. Experimental results show that *LoRA* can provide efficient nonlinear function implementation and achieve 23.33× and 2.18× average performance improvements compared to an STM32H750 microcontroller unit and a state-of-the-art (SOTA) CGRA. In addition, *LoRA* achieves a 2.13× average energy efficiency gain compared to the SOTA CGRA.

***Index Terms*—Reconfigurable Hardware, Nonlinear Function, Chebyshev Polynomial, Logarithmic Number System**

I. Introduction

The demise of Dennard scaling and the impending end of Moore's Law have spurred the rapid advancement of accelerators in various domains [11], [14], [28], [30]. Although these specialized accelerators can achieve optimized performance and energy efficiency, they are inflexible. In addition, such a specialization leads to dark silicon, since many accelerators remain underutilized across diverse workloads.

To overcome the above limitation, the Coarse-Grained Reconfigurable Architecture (CGRA) provides a balanced solution for performance, efficiency, and post-silicon reconfigurability. Consequently, CGRA has attracted considerable attention from both academia [16], [22], [26], [27], [38], [42], [51], [56], [61], [64], [70], [78] and industry [24], [59].

*Co-first author.

This work is supported by National Science and Technology Major Project (2021ZD0114701). We thank all the reviewers for their time and thoughtful feedback on this paper.

Conventionally, a CGRA consists of word-level reconfigurable processing elements (PEs) connected by reconfigurable interconnects. Given a targeted computation-intensive loop kernel, the compiler transforms it into a Data Flow Graph (DFG) and then maps it to the CGRA for execution. However, as shown in Table I, with the rapid development of applications, the applicability of conventional CGRA becomes limited. This is because these emerging applications often contain nonlinear functions (e.g., elementary functions) that current CGRAs can not support. As a result, most existing CGRAs offload these computations on the host CPU, leading to performance degradation.

TABLE I
Overview of Nonlinear Functions in Different Applications

Categories	Nonlinear Functions	Typical Application
Activation Function	Sigmoid(x)=$(1+e^{-x})^{-1}$, GELU(x)= $\frac{x}{2}\times(1+tanh(\sqrt{\frac{2}{\pi}}(x+0.044715x^3)))$, SiLU($x$)= $x\times$ Sigmoid(x), etc.	Deep Neural Network [34], [36], [65] Large Language Model [20], [37], [57]
Trigonometric Functions	sin(x), cos(x), sinh(x), cosh(x), arcsin(x), etc.	Discrete Cosine Transform [1] Synthetic Aperture Radar [13]
Others	$log_b(x)$, x^y, etc.	Digital Signal Processing [49]

As shown in [21], a simple way to enhance CGRA applicability is by adding off-the-shelf Intellectual Properties (IPs) for each nonlinear function [19], [21]. However, this approach is inefficient because: **(1)** hardware overhead grows with the number of IPs, and **(2)** while well-developed IPs for elementary functions provide accurate results, many real-world applications are error-tolerant [13], [18]. For example, PICACHU [56] uses Taylor expansions to approximate four nonlinear functions (*exp*, *log*, *sin*, *cos*) in large language models (LLMs), but requires more PEs as approximation accuracy increases. This problem becomes more significant when a compound nonlinear function is involved. NX-CGRA [54] transforms complex nonlinear operations into basic arithmetic (e.g., addition, multiplication) via its compiler, executing them in parallel on its Very-Long-Instruction-Word-based CGRA. While flexible, this approach demands more resources, resulting in higher latency and power consumption. Consequently, there is a great demand to design a reconfigurable and general-

979-8-3315-5065-3/26/$31.00 ©2026 IEEE
DOI 10.1109/ISCA66397.2026.00068

purpose approximating unit within the CGRA, making it applicable for versatile nonlinear functions.

Prior research proposes various methods for approximating nonlinear functions, including LUT-based and polynomial-based approaches [4], [7], [35], [56], [80]. In this paper, we combine polynomial and LUT approaches to approximate nonlinear functions. However, designing a flexible approximating unit for a general-purpose CGRA faces two challenges: **(1)** The approximation approach should be universal to be adopted by versatile nonlinear functions with user-defined input ranges, rather than being limited to a subset of functions [23], [56]. **(2)** The overhead of multiply-addition operations required by high-degree approximation polynomials cannot be ignored [4], [50], requiring an efficient polynomial computing approach.

To address the above challenges, this paper presents several contributions, where all the source codes are available from[1]:

- **Algorithm.** Based on the Chebyshev polynomial, we propose an efficient piecewise approximation algorithm, considering: **(1)** algebraic properties of the targeted function; **(2)** properties of the underlying architecture; **(3)** breakpoints for the nonuniform segmentation.
- **Architecture.** We integrate a lightweight reconfigurable functional unit *XCore* to CGRA for the nonlinear function, where the results of the proposed approximation algorithm configure this unit. By adopting the logarithmic number system (LNS), several nonlinear functions and the expensive multiply-addition operations in high-degree polynomials can be simplified.
- **Framework.** We propose *LoRA*[2], an end-to-end framework with architectural and compiler support for efficient nonlinear function computation in CGRA. By leveraging the proposed algorithm-hardware co-optimization, our CGRA is future-proof and can handle a wide range of nonlinear functions in upcoming applications.

Paper Organization: The background and motivation are shown in Section II, followed by the error analysis in Section III. Next, we introduce the architecture of *XCore* in Section IV and the Chebyshev-based approximation algorithm in Section V, respectively. An overview of the *LoRA* framework is provided in Section VI. The evaluation setup and results are presented in Section VII and Section VIII. Finally, we make the conclusion in Section IX.

II. Background and Motivation

This section provides an overview of the related technologies and highlights our motivation.

A. Nonlinear Functions in Hardware

Nonlinear functions are typically implemented by three categories of approaches.

1) Iterative-based Approach: Based on the selected coordinate system and rotation/vectoring mode, the Coordinate Rotation Digital Computer (CORDIC) algorithm approximates the targeted function iteratively [10], [47] with a low hardware overhead thanks to its simple operations (e.g., *add* and *shift*). However, CORDIC requires more iterations to achieve better accuracy, resulting in a significant increase in operation latency. Besides, it only supports a limited input range, and some versions require additional processing before/after rotation.

2) LUT-based Approach: For the LUT-based approach [35], [77], the output values of the targeted function are precomputed and stored in memory. Then, the input values act as the index to address the memory for the required outputs. Although the LUT-based approach reduces computation latency and simplifies implementation, it requires a large memory to achieve high accuracy. In addition, the LUT-based approach must trade off area overhead against the supported input range, thereby bounding the approximation to a specific region of the function, which limits its applicability.

The polynomial-based approach approximates nonlinear functions, such as the Taylor series [21], [50], [56], using degree-*n* polynomials. However, it faces limitations: **(1) Computational Overhead:** High-degree polynomials require multiple multiply-add (MAD) operations. While Horner's rule [33] simplifies computation, the overhead remains substantial. For instance, a 6th-degree Taylor polynomial for the *exp* function requires at least 6 MADs [4]. **(2) Input Range:** Taylor series accuracy is high only near the expansion point [21], limiting its effectiveness for applications with diverse input ranges. Although PICACHU [56] decomposes ranges, the associated computational overhead is still significant.

Given the complexity and wide input range of the target function, using a single polynomial for approximation is challenging. A more flexible approach is piecewise approximation with different polynomials. To reduce computational overhead, most existing works focus on single and quadratic degree polynomials, balancing the number of intervals [4], [23], [25], [39], [45], [69]. This approach can be treated as the combination of LUT and polynomial approaches, but requires less memory than the LUT-based approach, since there are only *breakpoints* and *coefficients* of each polynomial to be stored. Consequently, the architecture is reconfigurable for different functions by adjusting the *coefficients*. However, there are several optimization challenges: **(1) Data format:** Many prior approaches are designed for specific data formats (e.g., fixed-point or floating-point) [25], [39], [45], while *LoRA* should support more formats for future flexibility. **(2) Nonuniform segmentation:** As demonstrated in [4], nonuniform segmentation concentrates more pieces where the function changes quickly or has high curvature, resulting in lower approximation errors than uniform partitioning. Hence, an efficient segment strategy is crucial when the number of intervals is limited.

Motivation: To address the limitations above, we propose a piecewise approximation using Chebyshev polynomials. In addition to maintaining flexibility for versatile nonlinear functions across various input ranges, a critical challenge is to

[1]LoRA is part of the Fusion framework, whose source code is available at: https://github.com/Dai-dirk/COFFA

[2]We name it *LoRA* because it is a Logarithmic-arithmetic-based Reconfigurable Architecture for executing nonlinear functions.

reduce the computational overhead of high-degree polynomials. Consequently, we employ the logarithmic number system to address this challenge.

B. Chebyshev Polynomials

Compared to the Taylor series, Chebyshev polynomials converge faster, minimize the maximum error over the entire interval, and require a lower degree for the same sup-norm error. They also avoid Runge oscillations and offer better numerical conditioning [13], [31]. As a result, Chebyshev polynomials are more efficient and stable for approximation.

Chebyshev polynomials of the first kind, defined as $T_n(x)$, form a key family of orthogonal polynomials on the interval [-1, 1]. An *n*-degree polynomial polynomial $T_n(x)$ is defined by the following recurrence relations:

$$\begin{cases} T_0(x) = 1 \\ T_1(x) = x \\ T_n(x) = 2xT_{n-1}(x) - T_{n-2}(x) \end{cases} \tag{1}$$

TABLE II
OPERATIONS IN ORDINARY AND LOGARITHMIC ARITHMETIC

Operation	Ordinary Arithmetic	Transforming Equation	Logarithmic Arithmetic
Multiplication	$x \times y$	$2^{log_2(x \times y)} = 2^{log_2 x + log_2 y}$	$log_2 x + log_2 y$
Division	$x \div y$	$2^{log_2(x \div y)} = 2^{log_2 x - log_2 y}$	$log_2 x - log_2 y$
Logarithmic	$log_b x$	$--$	$\frac{1}{log_2 b} \times log_2 x$
Power	x^y	$2^{log_2(x^y)} = 2^{y \times log_2 x}$	$y \times log_2 x$

C. Logarithmic Number System

The Logarithmic Number System (LNS) simplifies various arithmetic operations [46], [48], as shown in Table II. However, addition and subtraction are more complex in LNS, so we adopt a hybrid approach: addition and subtraction are handled using ordinary arithmetic, while other operations are performed in LNS. Expression (2) shows a polynomial with five terms, where c_i is the coefficient and x is the variable. This polynomial is transformed into LNS using the equations in Table II, resulting in the expression in (3). To compute each term, we first apply a logarithmic transformation to x, then perform addition and multiplication in LNS. Finally, the antilogarithm transformation is applied to $log_2 c_i + k_i \times log_2 x$, and the terms are summed using ordinary arithmetic. This approach simplifies the original power operations into addition and multiplication. For instance, $c_i x^9$ becomes $2^{log_2 c_i + 9 \times log_2 x}$, eliminating the need for numerous costly multipliers.

$$c_0 x^{k_0} + c_1 x^{k_1} + c_2 x^{k_2} + c_3 x^{k_3} + c_4 x^{k_4} \tag{2}$$

$$\begin{aligned} &2^{log_2 c_0 + k_0 \times log_2 x} + 2^{log_2 c_1 + k_1 \times log_2 x} + 2^{log_2 c_2 + k_2 \times log_2 x} \\ &+ 2^{log_2 c_3 + k_3 \times log_2 x} + 2^{log_2 c_4 + k_4 \times log_2 x} \end{aligned} \tag{3}$$

A related work [48] adopts the LNS to approximate nonlinear functions, but with the following limitations: **(1)** it only supports fixed point data; **(2)** it only supports Taylor series, lacking an efficient approximation algorithm.

III. ERROR ANALYSIS

A. Problem Formulation

Utilizing Chebyshev polynomials of at most *n* degree to approximate the given function $f(x)$ can be defined as equation (4), where c_i is a constant coefficient and $T_i(x)$ is the i-th order Chebyshev polynomial.

$$\tilde{f}(x) = \sum_{i=0}^{n-1} c_i T_i(x) \tag{4}$$

Next, the software approximation $\tilde{f}(x)$ is computed using LNS-based hardware to generate the result $\tilde{f}_{HW}(x)$. Consequently, the problem can be formulated as finding an efficient implementation of $\tilde{f}_{HW}(x)$ to minimize the absolute approximation error $\varepsilon(x)$, defined by equation (5).

$$\varepsilon(x) = |f(x) - \tilde{f}_{HW}(x)| \tag{5}$$

By introducing $\tilde{f}(x)$, the error can be decomposed and bounded using the triangle inequality as follows:

$$\varepsilon(x) = |f(x) - \tilde{f}_{HW}(x)| \leq \underbrace{|f(x) - \tilde{f}(x)|}_{\text{model error}} + \underbrace{|\tilde{f}(x) - \tilde{f}_{HW}(x)|}_{\text{implementation error}} \tag{6}$$

B. Analysis

Within the inequality (6), the overall error includes two parts: ① **Model error:** This error arises from approximating the target function $f(x)$ with a restricted function $\tilde{f}(x)$, depending on the segmentation strategy and polynomial degree. When this error equals zero, it means the software approximation result ($\tilde{f}(x)$) is exact. ② **Implementation error:** This error results from using LNS to implement $\tilde{f}(x)$, influenced by the logarithmic and antilogarithmic transformations. When this error equals to zero, it means the hardware implementation ($\tilde{f}_{HW}(x)$) perfectly matches the software result ($\tilde{f}(x)$). Then, the model error determines the final accuracy. Subsequently, we introduce the detailed analysis of the implementation error.

Definition 1. **(hardware transformation)** Given a variable x, specific hardware is required to transform it into $log_2 x$. We define the hardware transformation as $log_2 x = log_2' x + \delta$, where $log_2' x$ is the accurate result and δ is the transformation error. Similarly, the hardware-based antilogarithm transformation introduces an error, denoted as β.

Consequently, the transformation from expression (2) to (3) can be rewritten in equation (7). It's worth noting that c_i is a constant, so the accurate value of $log_2' c_i$ can be precomputed.

$$\begin{aligned} c_i x^{k_i} &= 2^{log_2' c_i + k_i \times (log_2' x + \delta)} + \beta = 2^{log_2' c_i x^{k_i} + k_i \delta} + \beta \\ &= c_i x^{k_i} 2^{k_i \delta} + \beta \approx c_i x^{k_i} (1 + ln2 \times k_i \delta) + \beta \end{aligned} \tag{7}$$

Based on equation (7), the key factors for minimizing implementation error are the errors in the logarithmic and antilogarithm transformations. Fortunately, the architecture of the logarithmic converter has been well studied in other literature [41], [43], [82], and can be adopted by *LoRA*.

IV. XCore Architecture

A. Data Format

XCore employs a hybrid number system, combining ordinary arithmetic and LNS formats. This lets *XCore* leverage both advantages: simple addition/subtraction in ordinary arithmetic and more complex operations (e.g., x^y) in LNS. The data type in *LoRA* is 32-bit, supporting fixed-point, floating-point (FP32), and LNS formats, as in Fig. 1. For flexibility, the fixed-point data allows programmable precision by adjusting the fraction width. Since LNS is undefined for zero and negative values, the absolute value is used for logarithmic transformation, with *zero* and *sign* encoded in additional bits.

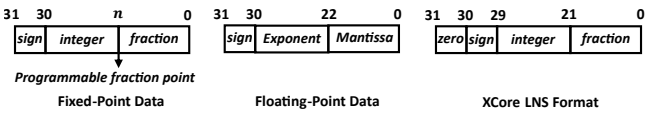


Fig. 1. The supported data formats by *XCore*.

B. Architecture Overview

By leveraging the hybrid number system, *XCore* can support various operations based on the configuration, mainly including four modes: **(1) Multiplication and division:** $x \times y$ and $x \div y$. **(2) Power:** x^y. **(3) Logarithmic:** $log_b x$. **(4) Polynomial:** *XCore* can support the polynomial with up to six terms, including five terms with variable (i.e., expression (3)) and one constant term (bias). Overall, as shown in Fig. 2(a), *XCore* has five stages, separated by registers. The following subsections outline these stages in the order of execution.

C. Pre-Process Stage

Since *LoRA* uses a piecewise approximation, each sub-interval polynomial has its own parameters: the coefficient ($log_2 c_i$), degree (k_i), and bias. As shown in expression (3), the coefficient $log_2 c_i$ is pre-computed and stored. Specifically, the bias is the polynomial term without the variable x. It can be a constant or the input y, which comes from another functional unit. During polynomial computation, the input variable x is compared with the *breakpoints* in this stage to address the LUT for the required parameters.

D. LOG Stage

In this stage, the input variable x is transformed into $log_2 x$ by the logarithmic converter. When computing $x^y = 2^{y \times log_2 x}$, the input variable y in fixed or floating point should multiply $log_2 x$ in LNS. Hence, the y is transformed into the LNS format (i.e., $y_{Q10,22}$). We present the logarithmic converter below, as it directly affects the implementation error.

***Logarithmic Converter Architecture*:** Following reference [46], the logarithm of a fixed-point format data x_1 can be defined by equation (8). Here, m is the most significant bit detected by the leading-one detector (LOD). Subsequently, the fraction part f can be determined based on m.

$$|x_1| = 2^m(1+f) \rightarrow log_2(|x_1|) = m + log_2(1+f), 0 \leq f < 1 \tag{8}$$

Similarly, the logarithmic transformation for a single-precision floating-point data x_2 can be defined by equation (9).

$$\begin{aligned} &x_2 = (-1)^s \cdot 2^{E-127} \cdot (1+f) \rightarrow \\ &log_2(|x_2|) = E - 127 + log_2(1+f), 0 \leq f < 1 \end{aligned} \tag{9}$$

Hence, the logarithmic transformation focuses on approximating $log_2(1+f)$, which has a small input range. The architecture of the logarithmic converter within *XCore* is shown in Fig. 2(b). For a fixed-point input x, the *LOD* unit detects the MSB and isolates f from the absolute value of x. For the floating-point input x, the f can be extracted directly. The *APP* unit then generates the approximation of $log_2(1+f)$, which is concatenated with other parts to form the LNS format. In this paper, we employ a state-of-the-art (SOTA) exploration framework to generate the *APP* unit [82] that achieves the desired accuracy while minimizing hardware overhead through an optimized piecewise-linear (PWL) approximation strategy.

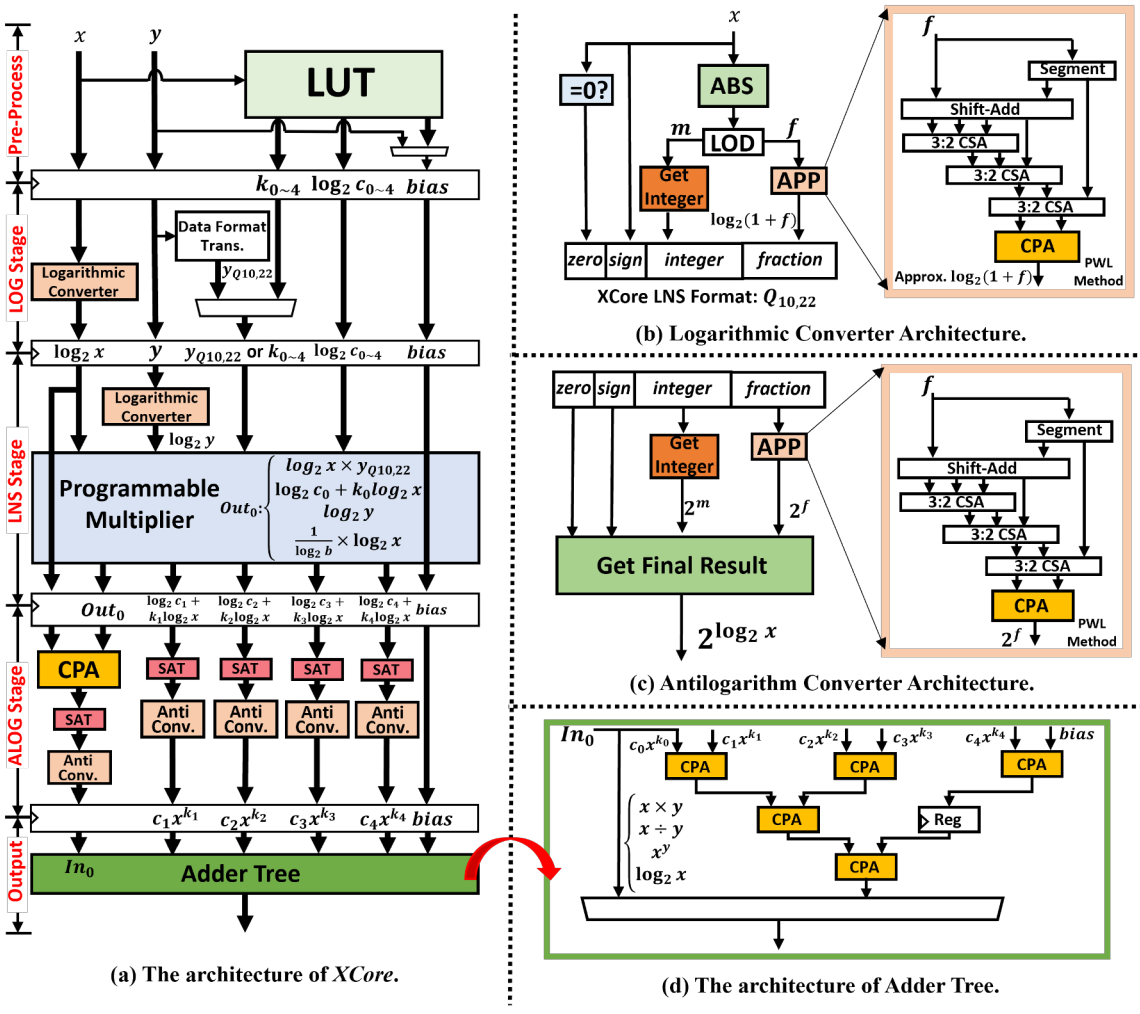
E. LNS Stage

At this stage, a programmable 30b×30b multiplier can be configured to perform various operations. It uses five Booth encoders to generate partial products (30b×6b each), which are then summed by adders. Based on the operation modes, the different operations include: **(1) Multiplication and division:** In this mode, the input variable y is transformed into $log_2 y$ for the next stage by the logarithmic converter, without multiplication. Hence, the design choice of transforming y in this stage is that the *APP* unit in the logarithmic converter can reuse the idle adders in the multiplier, thereby reducing overhead. **(2) Power and Logarithmic:** With a 32-bit LNS format (2-bit *sign* and *zero* flags), the multiplier performs 30b×30b multiplication to compute $y \times log_2 x$ or $\frac{1}{log_2 b} \times log_2 x$. **(3) Polynomial:** For 32-bit data, only 6 bits are needed for k_i to cover the exponent range. Consequently, the multiplier is configured to perform five 30b×6b multiplications and five additions to generate each $log_2 c_i + k_i log_2 x$. In Multiplication, Division, Power, and Logarithmic modes, the output is at out_0, while the outputs are across all the output ports in the Polynomial mode.

Discussion: *Why maximum six terms*? Considering both 30b×30b and 30b×6b multiplications are required by x^y, $log_b x$, and polynomials in different modes, it's area-efficient to decompose the one 30-bit operand into five 6-bit operands by reusing most of the hardware resources. As a result, there are five polynomial terms with the variable x input, along with one constant term (i.e., bias) added in the output stage.

F. ALOG Stage

When performing multiplication or division, the CPA in this stage is used to compute $log_2 x \pm log_2 y$. Subsequently, the saturation units (SATs) detect saturation/overflow and align the data format. Finally, each generated result is transformed into the ordinary arithmetic system by the antilogarithm converter, whose architecture is demonstrated below.



(a) The architecture of *XCore*.

(b) Logarithmic Converter Architecture.

(c) Antilogarithm Converter Architecture.

(d) The architecture of Adder Tree.

Fig. 2. The architecture of *XCore*, which employs a hybrid number system to handle different operations.

***Antilogarithm Converter Architecture*:** Equation (10) defines the antilogarithm transformation, where m is the floor of $log_2 x$. Consequently, as shown in Fig. 2(c), the antilogarithm transformation focuses on the 2^f term. We use a similar PWL approach and a custom fine-tuning process to produce the optimized antilogarithm converter.

$$2^{log_2 x} = 2^{m+f} = 2^m \cdot 2^f, 0 \le f < 1 \tag{10}$$

G. Output Stage

When the target operations are Multiplication, Division, Power, and Logarithmic modes, the results are output directly in this stage. The adder tree shown in Fig. 2(d) is used in Polynomial mode to sum each polynomial term in the ordinary arithmetic system. Each Carry-propagate Adder (CPA) can be configured to perform fixed-point or floating-point addition, and one register is added at the output to improve timing performance. Overall, the latency of Multiplication, Division, Power, and Logarithmic modes is four cycles, and the latency of Polynomial mode is seven cycles.

V. Chebyshev-based Approximation Algorithm

A. Problem Formulation: Single Interval Approximation

As discussed in Section III-A, the software approximation process involves finding a polynomial $\tilde{f}(x)$ of degree $\le n$ that minimizes the error between $\tilde{f}(x)$ and the target function $f(x)$. For a given $f(x)$ with an input range [a, b], the first step is to sample $f(x)$ in this interval. Here, we set the maximum number of sample points to *m*, so there are *m* pairs of data $\{(x_i, f(x_i))|i = 1, ..., m\}$. For the sake of simplicity, we set $n = 5$, leading to a polynomial that can have up to six terms (one constant term), which can be computed directly by one *XCore*. Subsequently, the approximation process is as follows.

Step 1: Interval transformation. Based on equation (11), the interval transformation is performed to map the input range [a, b] to [-1, 1], since Chebyshev polynomials are naturally

defined and orthogonal on [-1, 1]. In addition, the inverse transformation is defined by equation (12).

$$x \in [a,b] \rightarrow x^{'} = \frac{2x-(b+a)}{b-a}, x^{'} \in [-1,1] \tag{11}$$

$$x^{'} \in [-1,1] \rightarrow x = \frac{(x^{'}+1)(b-a)}{2} + a, x \in [a,b] \tag{12}$$

Consequently, the m pairs of data is transformed into $\{(x_i^{'}, f(\frac{(x_i^{'}+1)(b-a)}{2}+a) | i = 1, ..., m\}$.

Step 2: Construct Chebyshev matrix. Since the approximation value $\tilde{f}(x)$ is the sum of Chebyshev polynomials, we can construct the Chebyshev matrix (V) as equation (13).

$$V = \begin{bmatrix} T_0(x_1^{'}) & T_1(x_1^{'}) & T_2(x_1^{'}) & T_3(x_1^{'}) & T_4(x_1^{'}) & T_5(x_1^{'}) \\ ... & ... & ... & ... & ... & ... \\ T_0(x_m^{'}) & T_1(x_m^{'}) & T_2(x_m^{'}) & T_3(x_m^{'}) & T_4(x_m^{'}) & T_5(x_m^{'}) \end{bmatrix} \tag{13}$$

Subsequently, the approximation value for each $x^{'}$ can be obtained as follows, where $c_{0\sim5}$ are constant coefficients.

$$V \cdot [c_0, ..., c_5]^T = \left[\tilde{f}(x_1^{'}), ..., \tilde{f}(x_m^{'})\right]^T \tag{14}$$

Consider the algebraic property: When the target function exhibits symmetry, exploiting its parity improves approximation efficiency. For example, there are only odd-order terms for an odd function, leading to a simplified V in equation (15).

$$V = \begin{bmatrix} 0 & T_1(x_1^{'}) & 0 & T_3(x_1^{'}) & 0 & T_5(x_1^{'}) \\ ... & ... & ... & ... & ... & ... \\ 0 & T_1(x_m^{'}) & 0 & T_3(x_m^{'}) & 0 & T_5(x_m^{'}) \end{bmatrix} \tag{15}$$

In addition, exploiting the parity can achieve a higher polynomial degree for the same number of terms, leading to improved accuracy and better numerical stability.

Step 3: Solved by least squares. To have a smoother and more stable approximation, we use the evaluation function in equation (16) to minimize the squared error.

$$minimize(\sum_{i=0}^{m-1}(f(\frac{(x_i^{'}+1)(b-a)}{2}+a) - \tilde{f}(x_i^{'}))^2) \tag{16}$$

Problem formulation: Finding coefficients $[c_0, ..., c_5]^T$ to form a polynomial and minimize the evaluation function. This problem can be solved using the least squares method.

Step 4: Coefficient transformation. After obtaining $[c_0, ..., c_5]^T$, the $\tilde{f}(x)$ can be transformed as a standard polynomial based on the recurrence equations shown in equation (1). The transformed standard polynomial is shown in equation (17), where $p_i^{'}$ is a constant coefficient for each term.

$$\tilde{f}(x^{'}) = \sum_{i=0}^{5} c_i T_i(x^{'}) = \sum_{i=0}^{5} p_i^{'} \times x^{'i} \tag{17}$$

It's worth noting that $p_i^{'}$ is obtained based on $x^{'}$. Hence, to obtain the required $\tilde{f}(x)$, the coefficient $p_i^{'}$ should be transformed based on equation (11). Finally, the required $\tilde{f}(x)$ can be obtained based on equation (18).

$$\tilde{f}(x) = \sum_{i=0}^{5} p_i^{'} \times (\frac{2x-(b+a)}{b-a})^i = \sum_{i=0}^{5} p_i x^i \tag{18}$$

Hardware Constraint: *XCore* sums all the terms in the ordinary arithmetic system to compute a polynomial. Therefore, when the target format is fixed-point, p_i must be constrained to avoid overflow. Accordingly, the constraints in inequality (19) are incorporated into the least-squares solving process, where $Q_{m,n}^{max}$ is the upper bound of the target fixed-point format.

$$|p_i||x^i|_{max} < |Q_{(m.n)}^{max}|, i = 0, 1, ..., 5 \tag{19}$$

The above steps demonstrate the core process of approximating $f(x)$ in a given interval [a, b].

B. Approximation Algorithm Flow

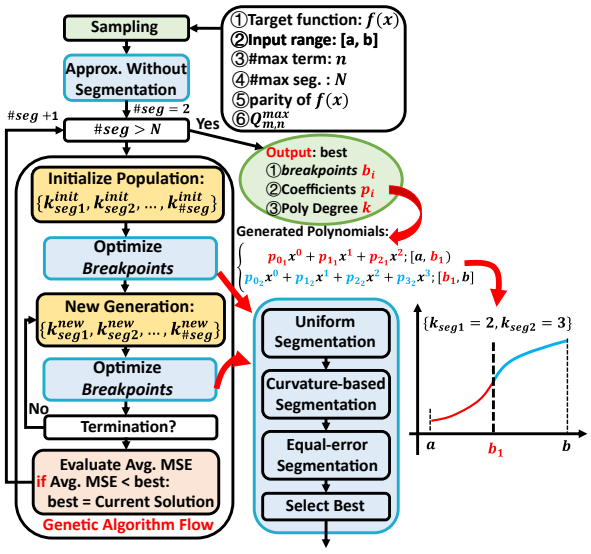


Fig. 3. The proposed Chebyshev-based approximation algorithm flow.

Fig. 3 shows the proposed Chebyshev-based approximation algorithm. Given a nonlinear function $f(x)$ with a user-defined input range [a, b], the algorithm first performs sampling, then approximates the function without segmentation while considering the maximum number of polynomial terms, the parity of $f(x)$, and the hardware constraint $Q_{m,n}^{max}$ for fixed-point target. The maximum term count determines the allowed polynomial degree. For symmetric functions, a higher degree can be supported with the same number of terms. For instance, with six terms, an asymmetric function supports degree five ($x^{0\sim5}$), whereas an odd function supports degree nine ($x^{1,3,5,7,9}$). It then starts from a two-interval approximation and gradually increases the number of sub-intervals until the maximum supported segmentation count (N) is reached.

However, a high-degree polynomial in every sub-interval may cause overfitting. To address this, we use a genetic algorithm to optimize degree allocation, as shown in Fig. 3. Each individual represents a candidate degree assignment for

all sub-intervals (i.e., $k_{seg1}, ..., k_{\#seg}$), while the *breakpoints* are determined by three segmentation strategies. After the algorithm terminates, the individual with the minimum average Mean Squared Error (MSE) is selected as the solution; if it outperforms solutions with other sub-interval counts, it is chosen as the *best* solution. The final output includes **(1)** breakpoints, **(2)** polynomial coefficients, and degree of each sub-interval. As shown in Fig. 3, the parameters highlighted in red are stored in the LUT of *XCore*. The algorithm can also explore polynomials with more than six terms, which can be computed by more than one *XCore*.

C. Curvature-Based Sampling

The sampling strategy affects how well the polynomial $\tilde{f}(x)$ approximates $f(x)$. Uniform sampling often performs poorly because it allocates points uniformly, even in regions where the function is smooth, while undersampling regions with rapid variation. To address this, we use a curvature-based strategy: starting with uniform samples on [a, b], we estimate curvatures through numerical differentiation and insert additional sample points in regions with higher curvature.

D. Multiple Segmentation Strategies

Given a target number of intervals, we employ three segmentation strategies: uniform, curvature-based, and equal-error. For each potential segment solution, the algorithm first determines the polynomial per sub-interval, and evaluates the approximation via maximum absolute error (MAE) or MSE.

1) Uniform Segmentation: The given interval [a, b] is divided into equal-length sub-intervals, assigning the same width to each segment regardless of the function's behavior.

2) Curvature-based Segmentation: The given interval [a, b] is divided into sub-intervals based on the curvature, which is computed during uniform sampling. Regions with higher curvature are assigned denser segments, while flatter regions receive fewer. Here, we assume the target sub-intervals count is *N*, and there are *m* uniform sampling points (i.e., $\{x_i | i = 1, ..., m\}$), each of which has a curvature $\kappa(x_i)$. In addition, Δx is defined as the uniform spacing between adjacent points. Then, the discrete cumulative curvature can be defined as equation (20), where W_m is the cumulative value for [a, b].

$$W_k = \sum_{i=1}^{k} \kappa(x_i)\Delta x, k = 1, ..., m \tag{20}$$

Subsequently, we can find the *breakpoints* $\{x_{b1}, ..., x_{bN-1}\}$ such that each interval contains an equal amount of cumulative curvature, which is $\frac{W_m}{N}$.

3) Equal-error Segmentation: The core idea here is to partition the original interval into sub-intervals such that the MAE within each interval is approximately the same. By balancing the local errors, the error distribution is more uniform, thereby achieving a near-optimal overall approximation accuracy [68].

Fig. 4 illustrates our equal-error segment process. Starting with curvature-based segmentation, breakpoints are iteratively optimized to minimize the MAE across all sub-intervals. A new breakpoint x_s^{j+1} is searched within the range between

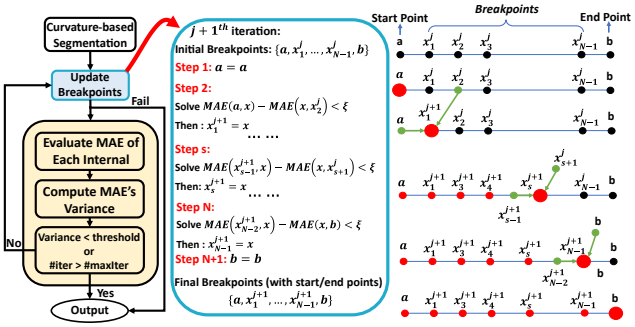


Fig. 4. The equal-error segmentation with the target sub-intervals count *N*.

its left point x_{s-1}^{j+1} (from the current iteration) and its right point x_{s+1}^{j} (from the previous iteration), ensuring the MAEs of the left and right sub-intervals are close. Then, the x_s^{j+1} can be determined by solving the inequality $MAE(x_{s-1}^{j+1}, x) - MAE(x, x_{s+1}^{j}) < \xi$ using Brent's method [9], where ξ is the tolerance value. If the root can not be found, the process is terminated. After updating the breakpoints, the MAE variance across all sub-intervals is evaluated. If the variance is below the threshold, the final breakpoints are returned.

VI. LoRA Framework

A. End-to-End Framework: Hardware

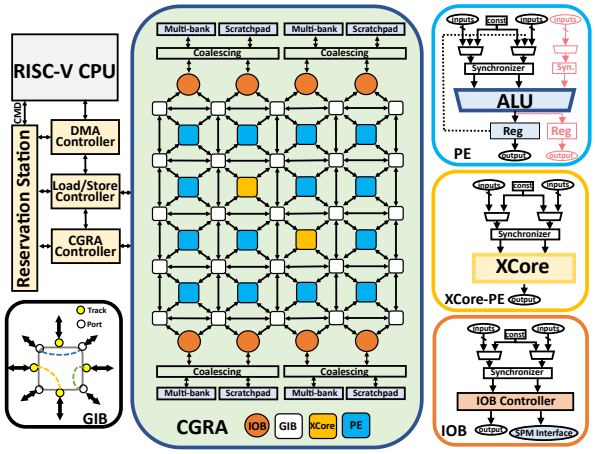


Fig. 5. The architecture of *LoRA* SoC, which includes a RISC-V CPU and a heterogeneous CGRA.

Built upon Chipyard [3], the *LoRA* SoC includes a 5-stage in-order scalar RISC-V CPU (64-bit) *Rocket* [6], a heterogeneous CGRA, and other subsystems, as illustrated in Fig. 5. The CPU manages data, invokes the CGRA for acceleration, and handles code not executable on the CGRA. The reservation station caches the custom RoCC (Rocket custom coprocessor) instructions and dispatches them to each controller based on different dependencies. The load and store controller manages data transmission between the L2 cache and scratchpad memories (SPMs) via TileLink, implemented

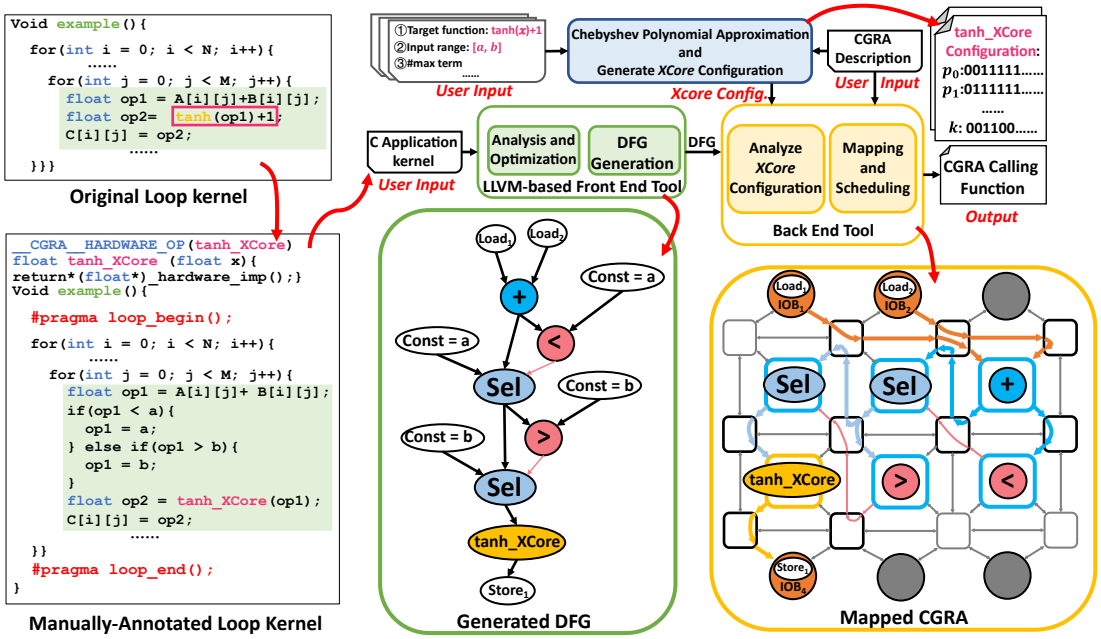


Fig. 6. The software toolchain of *LoRA*, which can support the custom nonlinear function implementation.

by the DMA controller. In addition, the CGRA controller configures the CGRA and monitors its execution status.

The CGRA is a spatial architecture similar to [16], [27], [66], [71], where each functional unit loads one configuration and executes all iterations, then switches the configuration for the next kernel. Each unit has its own configuration memory. The key components are: **(1) PE**, which is responsible for normal computation. Since partial-prediction [29] is used for the branch, an additional fine-grained input and output (shown in light red) are required for the select operation. **(2) *XCore*-PE,** which is responsible for complex operations, such as polynomials and x^y. **(3) IOB.** The input/output block (IOB) is responsible for loading or storing data from/to SPMs. Loop meta-information (e.g., loop entry/exit) is implicitly encoded in memory accesses and managed by the IOB controller [16], [72], [73]. For affine accesses, the IOB controller generates SPM addresses from its configuration (automatically derived from front-end analysis). For the non-affine access (e.g., $A[B[i]]$ or $A[i * i]$), addresses are computed at runtime by other units and fed via a second input. Hence, the IOB has two inputs, connecting to two GIBs for sufficient connectivity. **(4) GIB.** The general interconnection block (GIB) provides three types of flexible interconnect resources [62]: port-to-port, track-to-port, and track-to-track. The track-to-track connection follows the Wilton pattern [67], [75] and facilitates long-distance data communication. The back-end generates a configuration of the used GIB for the required connections. In addition, register-based synchronizers inside functional units ensure data synchronization, similar to [16], [71]. Our parameterized design allows configuring the CGRA size, PE type per position, and supported operations per PE.

B. End-to-End Framework: Software

Fig. 6 shows the software toolchain within *LoRA*. Loop kernels for CGRA execution are manually annotated with $\#pragma$ directives to identify the targeted kernel and custom nonlinear functions. The LLVM-based front-end tool then performs loop analysis, optimization, and DFG generation. Three additional steps support nonlinear functions: **(1) Manual Analysis.** Determine the type, input range, and data format of each nonlinear function. Since *XCore* can approximate compound nonlinear functions, it reduces the required computing resources. For example, the function $tanh(x) + 1$ in Fig. 6 or more complex ones like $sin(x) + cos(x)$ and $ln(sin(x))$ can all be approximated by a single *XCore*. **(2) Add Custom Function.** Nonlinear functions are defined as custom functions with the *__CGRA__HARDWARE_OP* annotation, ensuring special handling during DFG generation. As shown in Fig. 6, the original nonlinear function is replaced by the custom *tanh_XCore* function, which becomes a DFG node. In addition, to handle cases where the input may exceed the analyzed input range at runtime, the user can add corresponding computational constraints or range transformation (e.g., transform the input range to $[-\pi, \pi]$ for the trigonometric function) in the loop kernel. When such range transformations are not supported, the original function can be decomposed into combinations of primary functions (e.g., x^y, $log_b x$), with *XCore* supporting the entire input range. **(3) Generate Polynomials.** The Chebyshev-based algorithm generates polynomials for each nonlinear function, given the maximum supported sub-interval count (i.e., N in Fig. 3) of the CGRA. After that, the configuration of each nonlinear function is generated based

on the approximating polynomials, and the configuration will control the *XCore* to perform the corresponding computation.

Subsequently, the back-end tool collects the configuration of *XCores* and maps the DFG to the CGRA. A simulated annealing-based spatial mapping algorithm, similar to [16], [71], is used. Additionally, the back-end tool utilizes a memory partition algorithm [15], [17], [74] to allocate data to the multi-bank SPM and schedule bank-conflicting accesses into different time slots to prevent memory contention. Finally, the CGRA calling function is generated for the CPU to invoke the CGRA; it includes instructions to load configuration and data into the SPM, configure the CGRA, activate it for execution, and write results back to main memory. These custom instructions are sent to the reservation station via the RoCC interface. The original loop kernel is replaced with this function and compiled into an executable bare-metal file.

VII. Evaluation Setup

A. Comparison Methodology

The comparison is conducted in three levels, including: **(1) Algorithm level.** We study how different genetic-algorithm settings affect approximation runtime and final accuracy. **(2) Unit level.** We evaluate *XCore* by analyzing the trade-offs among converter accuracy, the maximum supported number of intervals and polynomial terms, and the resulting approximation error. We then compare *XCore* with prior nonlinear computation units, and further perform end-to-end accuracy evaluation to compare *LoRA* with PICACHU [56]. **(3) System level.** We first quantify the SoC overhead of integrating *XCore* into the CGRA. We then compare performance and energy efficiency between *LoRA* and PICACHU, and evaluate scalability by varying CGRA sizes and comparing against the STM32H750 MCU [2] (ARM Cortex-M7). It's worth noting that the approximation result of each nonlinear function involved in the evaluation is generated by a single *XCore*, which is configured by the software results.

TABLE III
The Benchmarks Used at System-level Evaluation

Benchmark	Application	Nonlinear Function
Mish	Activation Function [55], [56]	$Mish(x) = x \times tanh(ln(1+e^x))$
Logsigmoid		$Logsigmoid(x) = ln(\frac{1}{1+e^{-x}})$
Softmax		$Softmax(x_i) = \frac{\exp(x_i)}{\sum_{j=1}^{K}\exp(x_j)}, \quad i = 1, \ldots, K$
Softplus		$Softplus(x) = ln(1+e^x)$
Swiglu		$Swiglu(x) = SiLU(xW+B) \odot (xV+c)$ $SiLU(x) = \frac{x}{1+e^{-x}}$
Svm	Support Vector Machine	e^x
DCT	Discrete Cosine Transform [40]	$cos(x)$
KNN	K-Nearest Neighbors [60]	$\sqrt{x}$

B. Benchmarks

Eight nonlinear functions are used to evaluate *LoRA* at the unit level. At the system level, we select eight benchmarks with eleven loop kernels from diverse domains for comparison, as shown in Table III. The activation function benchmarks are derived from [55], [56], with matrix multiplication performed first to generate inputs for the nonlinear functions. These activation functions are commonly used in workloads like *Swiglu* in DeepSeek [20] and *Softmax* in most LLMs.

C. System-level Implementations

1) Hardware Implementation: We model three types of SoCs with different CGRAs for comparison: ① **Generic CGRA.** This baseline CGRA, similar to existing spatial CGRAs [16], [26], [27], [71], supports both fixed-point and floating-point formats but cannot handle nonlinear functions. It is used solely to evaluate the hardware overhead of adding nonlinear operation support. ② **PICACHU.** It includes two key technologies for efficient nonlinear function support: **(1)** Floating-point to Fixed-point (FP2FX) Conversion Module for exponential computations via Taylor expansions. **(2)** Operator fusion: Since Taylor expansions introduce a certain amount of MAD operations, supporting this common pattern through a specialized module in a single PE can reduce the required number of PEs. We re-implement these two modules and integrate them into the PE of Generic CGRA, thereby computing nonlinear functions as described in [56]. ③ **LoRA.** Enhances the Generic CGRA by integrating *XCore*-PE and adding MAD operations in PE to improve compute density.

TABLE IV
Benchmark Characteristics

Kernel	PICACHU		LoRA		Kernel	PICACHU		LoRA	
	#Node	#Edge	#Node (#XCore)	#Edge		#Node	#Edge	#Node (#XCore)	#Edge
Mish	36	45	11 (1)	13	Logsigmoid	34	43	13 (1)	15
Softmax_1	5	5	5 (0)	5	Softmax_2	18	22	8 (1)	7
Softmax_3	4	4	4 (0)	4	Softplus	34	43	14 (1)	16
Swiglu	37	47	15 (1)	17	Svm	21	25	11 (1)	11
DCT	33	46	21 (1)	24	KNN_1†	–	–	6 (1)	5
KNN_2	15	18	15 (0)	18					

† PICACHU cannot support $\sqrt{x}$, leaving it executed by CPU.

Subsequently, the CGRA size is set based on: (1) The maximum memory and computing node counts, since spatial CGRA is required to provide sufficient resources to accommodate the DFG. (2) The back-end tool's ability to find mapping solutions. Table IV shows the node count of the used kernels without loop unrolling. Consequently, to ensure a fair comparison, the size of each CGRA is 6×6, comprising 36 PEs and 12 IOBs with 12 SPM banks, each providing 4 KB of storage. Meanwhile, the size of L2 cache is 128KB. PICACHU and *LoRA* are heterogeneous designs, with only part of the PEs added to support the MAD operation. In this evaluation, the 36 PEs of *LoRA* include two *XCore*-PEs, a number determined by the *XCore* nodes in the DFG and the potential for further loop unrolling. Using custom *XCore* nodes for composite functions significantly reduces the DFG size in *LoRA*.

All the SoCs except *XCore* are modeled using Chisel [8], while the *XCore* is modeled by Verilog. The generated Verilog can be used for ASIC implementation or FPGA prototyping. We use the Synopsys Design Compiler to synthesize all the SoCs with the TSMC 40*nm* process library and compiled memory to estimate area and power consumption.

2) Software: The full-stack CGRA toolchain is implemented in C++, while the Chebyshev-based approximation algorithm is implemented in Python.

3) Simulation: We employ the Synopsys VCS-based simulator to verify correctness and measure performance.

VIII. Evaluation Result

A. Algorithm-Level Evaluation

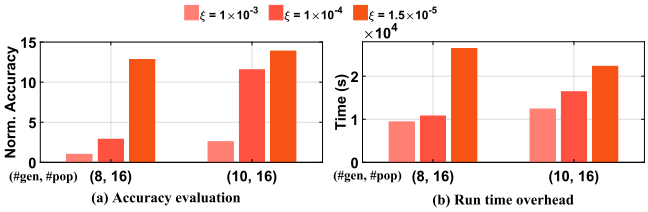


Fig. 7. The exploration of different algorithm settings.

Based on a set of nonlinear functions, we evaluate the impact of genetic algorithm settings, including: (1) the number of generations ($\#gen$) and population size ($\#pop$); (2) the tolerance value ξ for equal-error segmentation. Fig. 7 shows the normalized accuracy and average run time for one function. Our observations are: (1) A smaller ξ means the error distribution is more uniform, bringing results closer to the near-optimal accuracy, but increases computational overhead and run time. (2) More iterations improve results, but yield diminishing returns once the solution is near optimal. We also tried an exhaustive search, but with 6 segments and 6 terms, each segment has 6 possible degrees ($k = 0 \sim 5$), resulting in 6^6 possible degree allocations, making it infeasible to complete the exploration in a week. Therefore, despite its randomness, we find the genetic algorithm more suitable once the iteration count is sufficient. Based on our experience, we typically set $\xi = 1.5 \times 10^{-5}$, $\#gen = 10$, and $\#pop = 16$.

B. Unit-Level Evaluation

1) Trade-offs in XCore Design: As shown in inequality (6), approximation accuracy is affected by two factors: ① **Model error**, reducible by increasing segments or polynomial terms, and ② **implementation error**, determined by the accuracy of logarithmic/antilogarithmic converters. To study these trade-offs, we implement twenty *XCores* with different converter accuracy (quantified by the maximum absolute error) and segment counts. Using a set of nonlinear functions, Fig. 8(a)–(b) report their normalized area-delay-power products (ADPP) and average approximation accuracies (quantified by MSE). We observe that as converter accuracy and segment count increase, accuracy improves, but hardware cost also increases. We

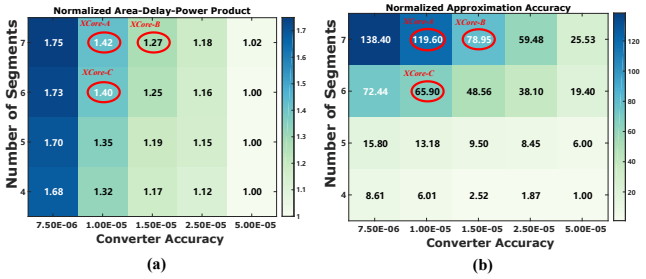


Fig. 8. The design space exploration of *XCore*, where the designs highlighted with a red circle are selected for evaluation.

therefore select three representative designs for the following unit-level evaluation (*XCore*-A∼C in Fig. 8), based on the following considerations: (1) When the number of segments exceeds 6, the accuracy improves significantly. Therefore, among the designs with six segments, *XCore*-C is selected for its superior area efficiency. (2) Among the designs with 7 segments, *XCore*-B is chosen because it achieves better accuracy than *XCore*-C while incurring a smaller area overhead. In addition, compared with *XCore*-C, *XCore*-A offers a significant accuracy improvement with only a modest area increase, leading to its selection. Subsequently, with 7 segments, we evaluate the impact of the number of polynomial terms.

TABLE V
The Evaluation on Different Numbers of Polynomial Terms

	***XCore*: 4 terms**			***XCore*: 5 terms**			***XCore*: 6 terms**	
Converter Accuracy	0.004	0.00125	0.0005	0.000175	0.00006	2×10^{-5}	**A** 1×10^{-5}	**B** 1.5×10^{-5}
Norm. ADPP†	1.0	1.29	1.48	2.43	3.46	4.87	6.95	6.40
Norm. Accuracy_HW†	1.0	17.3	79.4	771.9	6112.6	34961.4	93789.1	51629.2
Norm. Accuracy_SW†	1.0			58.4			237.1	

† Each metric is normalized to the respective result of the first 4-term *XCore*.

Table V shows the following observations: (1) Software (SW) results indicate that higher-degree polynomials with more terms improve accuracy. (2) As discussed in Section IV-E, when the number of terms is six, a 30-bit operand is decomposed into five 6-bit operands to store each k_i. Therefore, when the number of terms is reduced to five or four, the operand width should be reduced to 24-bit and 18-bit, respectively, to avoid wasting resources, thus reducing the LNS format width. As a result, achieving high converter accuracy in the reduced LNS width is challenging. For four and five terms, we evaluate six *XCores*. Despite the increased hardware overhead as the number of terms and converter accuracy rise, the gains in approximation accuracy are significant. Specifically, compared to the best result with five terms, *XCore-A* achieves a 2.68× accuracy improvement at the cost of 1.43× ADPP, making the six-term configuration a good design choice.

2) Function Approximation Analysis: In Table VI, we compare *XCore* with prior works using evaluation metrics such

TABLE VI
THE COMPARISON OF NONLINEAR COMPUTATION UNITS

Work	Func.	Range	AAE	sq-AAE	MSE	Domain
[76]	Sigmoid	[-8, 8]	1.70×10^{-3}	2.89×10^{-6}	–	AI
[32]		[-8, 8]	3.40×10^{-4}	1.16×10^{-7}	2.55×10^{-8}	
[4]		[-8, 8]	–	6.50×10^{-9}	–	
XCore-A		[-8, 8]	**3.73×10^{-6}**	**1.39×10^{-11}**	**2.36×10^{-11}**	
XCore-B		[-8, 8]	5.54×10^{-6}	3.07×10^{-11}	5.87×10^{-11}	
***LoRA*-SW-7-seg**		[-8, 8]	1.95×10^{-6}	3.80×10^{-12}	1.06×10^{-11}	
XCore-C		[-8, 8]	8.34×10^{-6}	6.95×10^{-11}	1.42×10^{-10}	
***LoRA*-SW-6-seg**		[-8, 8]	7.02×10^{-6}	4.92×10^{-11}	1.31×10^{-10}	
[25]	Tanh	[-8, 8]	3.74×10^{-4}	1.40×10^{-7}	5.55×10^{-7}	AI
[4]		[-8, 8]	–	1.02×10^{-8}	–	
XCore-A		[-8, 8]	**6.48×10^{-6}**	**4.20×10^{-11}**	**1.41×10^{-10}**	
XCore-B		[-8, 8]	9.34×10^{-6}	8.73×10^{-11}	3.12×10^{-10}	
***LoRA*-SW-7-seg**		[-8, 8]	4.98×10^{-6}	2.48×10^{-11}	7.94×10^{-11}	
XCore-C		[-8, 8]	3.63×10^{-5}	1.32×10^{-9}	2.01×10^{-9}	
***LoRA*-SW-6-seg**		[-8, 8]	3.62×10^{-5}	1.31×10^{-9}	2.02×10^{-9}	
[79]		[-4, 4]	–	–	1.28×10^{-7}	
[4]		[-4, 4]	–	–	**7.10×10^{-11}**	
XCore-A		[-4, 4]	7.61×10^{-6}	5.78×10^{-11}	9.42×10^{-11}	
XCore-B		[-4, 4]	1.06×10^{-5}	1.12×10^{-10}	2.04×10^{-10}	
***LoRA*-SW-7-seg**		[-4, 4]	4.88×10^{-6}	2.38×10^{-11}	5.31×10^{-11}	
XCore-C		[-4, 4]	1.05×10^{-5}	1.10×10^{-10}	1.41×10^{-10}	
***LoRA*-SW-6-seg**		[-4, 4]	1.01×10^{-5}	1.01×10^{-10}	1.58×10^{-10}	
[79]	GELU	[-8, 8]	–	–	1.76×10^{-8}	AI
[4]		[-8, 8]	–	9.09×10^{-9}	–	
XCore-A		[-8, 8]	**2.20×10^{-5}**	**4.82×10^{-10}**	**8.64×10^{-10}**	
XCore-B		[-8, 8]	2.27×10^{-5}	5.15×10^{-10}	9.26×10^{-10}	
***LoRA*-SW-7-seg**		[-8, 8]	2.13×10^{-5}	4.54×10^{-10}	8.24×10^{-10}	
XCore-C		[-8, 8]	3.03×10^{-5}	9.16×10^{-10}	1.69×10^{-9}	
***LoRA*-SW-6-seg**		[-8, 8]	2.97×10^{-5}	8.84×10^{-10}	1.60×10^{-9}	
[25]	Swish	[-8, 8]	4.90×10^{-4}	2.40×10^{-7}	4.51×10^{-7}	AI
XCore-A/C		[-8, 8]	**1.25×10^{-5}**	**1.56×10^{-10}**	**2.37×10^{-10}**	
XCore-B		[-8, 8]	1.49×10^{-5}	2.22×10^{-10}	3.83×10^{-10}	
***LoRA*-SW**†		[-8, 8]	1.08×10^{-5}	1.17×10^{-10}	1.63×10^{-10}	
[80]	Softplus	[-8, 8]	–	–	1.26×10^{-6}	AI
[32]		[-8, 8]	9.33×10^{-4}	8.70×10^{-7}	1.81×10^{-7}	
XCore-A		[-8, 8]	**6.46×10^{-6}**	**4.17×10^{-11}**	**7.95×10^{-11}**	
XCore-B		[-8, 8]	8.81×10^{-6}	7.77×10^{-11}	1.53×10^{-10}	
***LoRA*-SW-7-seg**		[-8, 8]	2.90×10^{-6}	8.42×10^{-12}	2.75×10^{-11}	
XCore-C		[-8, 8]	7.77×10^{-6}	6.04×10^{-11}	1.08×10^{-10}	
***LoRA*-SW-6-seg**		[-8, 8]	4.22×10^{-6}	1.78×10^{-11}	5.33×10^{-11}	
[10]	arcsinh	[-19.4, 19.4]	**8.91×10^{-6}**	**7.94×10^{-11}**	–	Target tracking, DSP
XCore-A		[-19.4, 19.4]	1.67×10^{-5}	2.78×10^{-10}	4.14×10^{-10}	
XCore-B		[-19.4, 19.4]	1.82×10^{-5}	3.30×10^{-10}	5.00×10^{-10}	
***LoRA*-SW-7-seg**		[-19.4, 19.4]	1.60×10^{-5}	2.55×10^{-10}	3.82×10^{-10}	
XCore-C		[-19.4, 19.4]	3.79×10^{-5}	1.43×10^{-9}	1.99×10^{-9}	
***LoRA*-SW-6-seg**		[-19.4, 19.4]	3.73×10^{-5}	1.39×10^{-9}	1.96×10^{-9}	
[44]	Sin	[0, 1]	1.20×10^{-3}	1.44×10^{-6}	—	Image processing, DSP
[32]		$[-\pi, \pi]$	2.09×10^{-3}	4.37×10^{-6}	9.82×10^{-7}	
[10]		$[-\pi, \pi]$	**3.24×10^{-7}**	**1.05×10^{-13}**	–	
XCore-A/C		$[-\frac{\pi}{2}, \frac{\pi}{2}]$	1.03×10^{-6}	1.07×10^{-12}	2.18×10^{-12}	
XCore-B		$[-\frac{\pi}{2}, \frac{\pi}{2}]$	1.81×10^{-6}	3.28×10^{-12}	7.05×10^{-12}	
***LoRA*-SW**†		$[-\frac{\pi}{2}, \frac{\pi}{2}]$	1.86×10^{-9}	3.47×10^{-18}	4.58×10^{-18}	
[10]	$\sqrt{x}$	[0, 63.75]	9.59×10^{-6}	9.20×10^{-11}	–	Image processing, DSP
XCore-A/C		[0, 63.75]	**6.41×10^{-6}**	**4.11×10^{-11}**	**7.39×10^{-11}**	
XCore-B		[0, 63.75]	1.11×10^{-5}	1.24×10^{-10}	2.22×10^{-10}	

† The software accuracies for 6 and 7 segments are identical, as the number of segments in the approximation result does not reach the hardware limit.

as average absolute error (AAE), squared AAE (sq-AAE), and MSE, with blue highlights indicating superior performance. While many prior methods are limited to fixed-point or low-precision formats, *XCore* offers distinct advantages: **(1)** It provides high-quality approximations across a range of complex nonlinear functions, even when compared to the CORDIC-based method [10] that aims to provide accurate results. **(2)** It is a general-purpose solution, supporting diverse nonlinear functions. Additionally, the software approximations (i.e., $\tilde{f}(x)$) from our Chebyshev-based algorithm, *LoRA*-SW, show that *XCore*'s approximations closely match software results, with minimal implementation error, as defined in inequality (6). However, an exception is the *sin* function, where *XCore* lags behind the software MSE due to the logarithmic converter's accuracy limitation, with an error bound around 1×10^{-5}. Thus, the *XCore* approximation reaches its error bound within the order of magnitude compared to the software's double-precision result.

TABLE VII
THE COMPARISON OF APPROXIMATION UNITS

Design	Huicore [10]	Flex-SFU [4]	*XCore* A	*XCore* B	*XCore* C	Design	PACE [53]	*XCore* 3,16
Tech.	28nm	28nm	40nm			**Tech.**	12nm	40nm
Area (μm^2)	153000	22915.6▲	78362.8	71728.7	77373.3	**Area** ($\#gate$)	∼32000	28740
Power (mW)	78.083	1.6	17.37	16.12	17.16	**Power** (mW)	No Available	11.7
Freq.	2GHz	500MHz	510MHz	485MHz	510MHz	**Freq.**	1GHz	600MHz
latency (cycles)	≥ 20†	11	mode❶-❸: 4 mode❹: 7			**latency (cycles)**	3	4/6*
Format	Fixed: $Q_{6,26}$ FP32	INT8/16/32 FP8/16/32	Fixed:$Q_{Programmable}$ FP32			**Format**	FP32/16 BF16	Fixed:Q_{any} FP32
x^y	support	×	support			x^y	×	support

▲ Excludes the MAD units.
† It only reports the number of required iterations.
* Latency is reduced due to fewer adders in the output stage.

Among the previous works listed in Table VI, only Flex-SFU [4] and huicore [10] aim to support versatile nonlinear functions like *XCore*. We further compare *XCore* with these designs in Table VII, with the following key observations: **(1)** Compared to huicore, *XCore* achieves lower latency and hardware overhead, while supporting a programmable fixed-point format. Additionally, *XCore* can approximate composite functions in one step, unlike the cascading approach used by CORDIC-based methods. **(2)** Flex-SFU uses a piecewise quadratic approximation, requiring two MAD operations. As a result, its area will be comparable to *XCore* once the MAD units are added. Moreover, *XCore* offers better approximation and supports binary operations (e.g., x^y).

In addition, we compare *XCore* with the latest work PACE [52], [53] from both algorithm and hardware perspectives: **(1) Algorithm.** Both *XCore* and PACE use Chebyshev polynomials for piecewise approximation, but *XCore* fully utilizes the target function's algebraic properties, offering superior approximation. For example, when approximating $cos(x)$, considering parity reduces the MAE by 148× compared to not considering parity. Moreover, the equal-error segmentation in *LoRA* provides more stable results. **(2) Hardware.** PACE simplifies floating-point multiplication by scaling data to integers. In contrast, *XCore* employs logarithmic arithmetic, simplifying complex operations and supporting both fixed and floating-point formats, as well as operations like x^y. Table VII compares *XCore* and PACE with the same supported segments ($\#seg$) and polynomial terms ($\#term$). While both support 3-term polynomials, PACE only handles $ax^3 + bx^2 + cx + d$, whereas *XCore* allows more complex form ($ax^{k_1} + bx^{k_2} + cx^{k_3} + d$). Additionally, we evaluate 3-term *XCore* approximation results on the same DNN models as in the PACE paper. The results show that *XCore*'s errors in EfficientNet and MobileNetV3 are 0.002% and 0.006%, lower than the smallest error (0.01%) reported in [53].

3) End-to-End Accuracy Analysis: *LoRA*-A/B/C are compared with the PICACHU algorithm [56] in three applications.

TABLE VIII
THE ACCURACY EVALUATION ON DCT APPLICATION

	Quantization = 0.1			Quantization = 0.75			Quantization = 1.5		
	MSE(↓)	PSNR(↑)	Compress Rate(↑)	MSE(↓)	PSNR(↑)	Compress Rate(↑)	MSE(↓)	PSNR(↑)	Compress Rate(↑)
Baseline (FP32)	3.231	43.065	2.979	35.629	32.755	10.795	68.010	30.009	17.073
LoRA-A/C	0.0	+ 0.001	0.0	- 0.001	0.0	+ 0.001	0.0	0.0	+ 0.128
LoRA-B	0.0	+ 0.001	0.0	- 0.001	0.0	+ 0.001	- 0.001	0.0	+ 0.128
PICACHU-3rd	+ 0.072	- 0.098	- 0.005	+ 0.116	- 0.014	- 0.012	+ 0.128	- 0.009	+ 0.111
PICACHU-4th	0.0	+ 0.001	0.0	0.0	0.0	+ 0.001	- 0.001	0.0	+ 0.128
PICACHU-5th	+ 0.001	0.0	0.0	0.0	0.0	- 0.002	0.0	0.0	+ 0.127

(1) Discrete Cosine Transform (DCT). The nonlinear function in DCT is *cos(x)*. We apply the approximation methods of *LoRA* and PICACHU to compute it and evaluate the DCT transformation in terms of MSE, Peak Signal-to-Noise Ratio (PSNR), and compression ratio after Huffman encoding, using the FP32 format. The baseline is the results generated by the AMD Ryzen 9 7945HX CPU. The evaluation results are shown in Table VIII, where the suffix of PICACHU denotes the order of its Taylor expansion. Results highlighted in blue indicate better performance, while those in red indicate worse performance compared to the baseline. We observe that a fourth-order or higher Taylor expansion is needed to match *LoRA*'s performance. Therefore, we set the Taylor expansion order to at least four for subsequent evaluations.

TABLE IX
THE ACCURACY EVALUATION ON DNNS

DNN▲	Data Format	Activation Function	Baseline (FP32)	LoRA			PICACHU	
				A	B	C	4th	5th
SE-ResNet [36]	FP32	Sigmoid	78.668%	-0.002%	-0.004%	-0.002%	-0.012%	-0.018%
	$Q_{16,16}$		78.668%	+0.002%	0.0%	+0.004%	-0.008%	-0.004%
EfficientNet [65]		Sigmoid, Swish	84.042%	-0.002%	0.0%	+0.008%	0.0%	-0.006%
MobileNetV3 [34]		HardSwish	75.642%	+0.002%	-0.008%	+0.002%	-0.010%	-0.010%

▲ Based on ImageNet dataset.

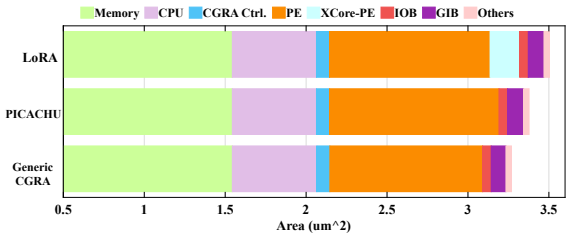


Fig. 9. The area breakdown for the SoCs.

(2) Deep Neural Network (DNN). All DNN experiments are conducted on an NVIDIA GeForce RTX 4090 GPU. The models are used in their original form without any fine-tuning, and the resulting outputs (in FP32 format) serve as the baseline. We then applied the approximation techniques *LoRA* and PICACHU to compute the activation functions, thereby replacing the original nonlinear functions, and evaluated the accuracy of these approximations, as shown in Table IX. The hardware data format is the target format during approximation for both *LoRA* and PICACHU. Based on the results, we observe that *LoRA* achieves better accuracy than PICACHU and even outperforms the baseline. While the *Hardswish* function does not involve nonlinear operations, it includes a polynomial that can be directly computed by *XCore*.

TABLE X
THE ACCURACY EVALUATION ON LLMS

Model	Activation Function	Approach	ARC-e (↑)	HS(↑)	CQ (↑)	CP (↑)
GPT2-XL [57]	GELU Softmax	Baseline	51.05%	50.89%	19.57%	76.00%
		LoRA-A/B/C	-0.04%	0.0%	0.0%	0.0%
		PICACHU-4th	-0.08%	0.0%	0.0%	-1.0%
		PICACHU-5th	-0.04%	0.0%	0.0%	0.0%
Mistral-7B [37]	Swish Softmax	Baseline	79.59%	81.07%	56.51%	92.00%
		LoRA-A/B/C	+0.59%	-0.07%	-1.39%	+1.0%
		PICACHU-4th	+0.59%	-0.08%	-1.47%	+1.0%
		PICACHU-5th	+0.59%	-0.07%	-1.39%	+1.0%
Mistral-7B-v0.3 [37]		Baseline	82.61%	82.93%	69.29%	93.00%
		LoRA-A/C	+0.04%	+0.01%	+0.08%	0.0%
		LoRA-B	+0.05%	+0.01%	+0.08%	0.0%
		PICACHU-4th	+0.04%	-0.02%	0.0%	0.0%
		PICACHU-5th	0.0%	0.0%	0.0%	0.0%
DeepSeek-7B [20]		Baseline	71.13%	76.15%	36.61%	84.00%
		LoRA-A/B/C	0.0%	0.0%	-0.08%	0.0%
		PICACHU-4th	0.0%	-0.01%	-0.16%	0.0%
		PICACHU-5th	0.0%	0.0%	-0.08%	0.0%

(3) Large Language Model (LLM). All LLM experiments are conducted on an NVIDIA A100 GPU. As with DNN evaluation, the GPU-generated results serve as the baseline. Then, we apply the approximation approaches of *LoRA* and PICACHU to compute the activation function (in fixed-point format $Q_{16,16}$) and evaluate the accuracy on several natural language processing tasks, including ARC-Easy [12], HellaSwag [81], CommonsenseQA [63], and COPA [58]. As shown in Table X, *LoRA* provides efficient approximation and even outperforms the baseline in some tasks.

Discussion: Accuracy improvement through approximation. Since the baseline model achieves less than 100% accuracy, its predictions inherently contain errors. Thus, approximations can act as corrective perturbations that partially offset these errors, rather than merely degrading precision. Our goal is a low-overhead hardware solution for complex nonlinear functions without sacrificing precision. As the evaluation shows, this goal is met: precision remains acceptable, and the need for highly accurate nonlinear computation is simplified.

Finally, *XCore*-C is selected for CGRA integration in the subsequent evaluations because it provides sufficient accuracy, moderate area and power consumption, and a higher frequency.

C. System-Level Evaluation

1) Hardware Overhead: We break down the area for each SoC in Fig. 9. Compared to the baseline Generic CGRA, PICACHU increases SoC and CGRA fabric area by 3.3% and 9.8%, respectively, due to the MAD operations and FP2FX module. Replacing two normal PEs with *XCore*-PEs, *LoRA*

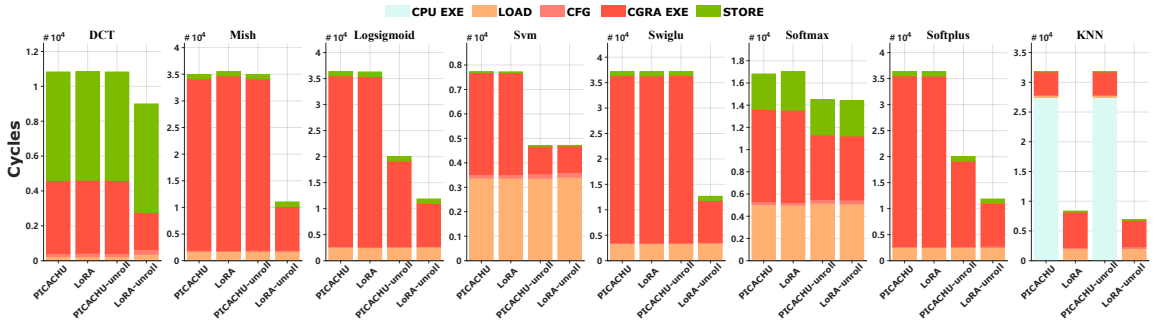


Fig. 10. The breakdown of the cycle count for each benchmark, where the cycle count includes: (1) the CPU executes the computation (CPU EXE); (2) loading data from external memory to SPM (LOAD); (3) configuring the CGRA (CFG); (4) the CGRA executes the computation (CGRA EXE); (5) storing the results back to the external memory (STORE). The -Unroll means that the loop is unrolled as many times as possible under hardware constraints.

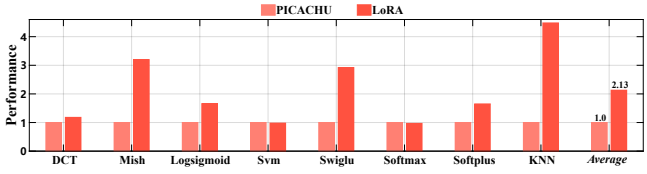


Fig. 11. Normalized energy efficiency over PICACHU, where the energy efficiency is computed based on the performance with loop unrolling.

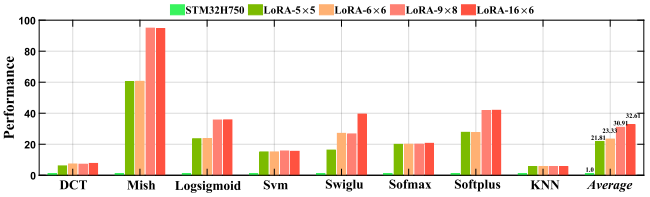


Fig. 12. Normalized performance over an STM32H750 MCU.

results in 3.7% and 10.1% area increases in SoC and CGRA fabric compared to PICACHU.

2) Performance and Efficiency: Since Generic CGRA cannot support nonlinear functions, the performance comparison is made between PICACHU and *LoRA*. Both PICACHU and *LoRA* operate at a maximum frequency of nearly 475 MHz, with the critical path in the floating-point adder of the normal PE. Hence, the metric used is the $\#cycle$, which represents the total cycles to execute each benchmark. Fig. 10 breaks down the cycle count per benchmark, revealing three observations: (1) PICACHU lacks general support for nonlinear operations, offloading $\sqrt{x}$ to the CPU and causing prolonged CPU execution. **This result shows that it's beneficial to add a general nonlinear operations support in the CGRA.** (2) PICACHU requires multiple PEs to perform MAD operations in the Taylor polynomials, thereby limiting the potential for loop unrolling. (3) In *DCT*, loop unrolling increases data loading and configuration time due to higher resource demand, but this overhead is negligible compared to execution time reduction. Overall, *LoRA* achieves an average performance improvement of 2.18× over PICACHU. Additionally, Fig. 11 shows that *LoRA* improves energy efficiency by 2.13× on average.

3) Scalability: We model several *LoRAs* with different sizes to study scalability and compare their performance against the STM32H750 MCU, which operates at 480 MHz (40*nm*) and utilizes the CMSIS-DSP library [5] for optimized performance. This MCU serves as a baseline because, as shown in the *KNN* benchmark, nonlinear operations unsupported by the CGRA are offloaded to the CPU. Comparing against a high-performance MCU thus highlights the potential gains of accelerating such operations on specialized hardware, quantifying the benefits of *LoRA* when nonlinear tasks are CPU-bound. In addition, both the MCU and our synthesized CGRA are based on similar process nodes (40nm), allowing for a fair hardware efficiency comparison. The operating frequencies of *LoRA*-5×5 (with 2 *XCores*) and *LoRA*-9×8/16×6 (with 3 and 4 *XCores*) are near 475 MHz and 470 MHz, respectively, demonstrating the timing scalability of our design. The performance metric here is runtime, which is defined as $\#cycle/frequency$. As shown in Fig. 12, *LoRA*-6×6 achieves a 23.33× average performance over the STM32H750. However, performance gains with increased hardware resources saturate due to two factors: (1) some loop kernels cannot be further unrolled, and (2) after full unrolling, execution time becomes dominated by data transmission (the memory wall). For example, in *DCT* and *Softmax* (Fig. 10), data transfer time dominates after unrolling. However, solving this problem is outside the scope of this work.

IX. Conclusion

To improve the applicability of reconfigurable architecture for versatile nonlinear functions, we propose *LoRA*, a comprehensive CGRA SoC framework. By leveraging the proposed Chebyshev-based approximation algorithm and logarithmic number system, *LoRA* can implement various nonlinear functions while maintaining high accuracy. In addition, the end-to-end framework with architectural and compiler support makes *LoRA* an efficient solution for accelerating a broad spectrum of applications. Therefore, we believe our design is future-proof and can significantly benefit computation-intensive systems.

REFERENCES

[1] "Discrete cosine transform," https://en.wikipedia.org/wiki/Discrete_cosine_transform.

[2] "STM32H750 Microcontroller Overview," https://www.st.com/en/microcontrollers-microprocessors/stm32h750vb.html.

[3] A. Amid, D. Biancolin, A. Gonzalez, D. Grubb, S. Karandikar, H. Liew, A. Magyar, H. Mao, A. Ou, N. Pemberton, P. Rigge, C. Schmidt, J. Wright, J. Zhao, Y. S. Shao, K. Asanović, and B. Nikolić, "Chipyard: Integrated Design, Simulation, and Implementation Framework for Custom SoCs," *IEEE Micro*, vol. 40, no. 4, pp. 10–21, 2020.

[4] R. Andri, E. Reggiani, and L. Cavigelli, "Flex-SFU: Activation Function Acceleration With Nonuniform Piecewise Approximation," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 44, no. 11, pp. 4236–4248, 2025.

[5] ARM Inc., "CMSIS-DSP Library," https://github.com/ARM-software/CMSIS-DSP.

[6] K. Asanović, R. Avizienis, J. Bachrach, S. Beamer, D. Biancolin, C. Celio, H. Cook, D. Dabbelt, J. Hauser, A. Izraelevitz, S. Karandikar, B. Keller, D. Kim, J. Koenig, Y. Lee, E. Love, M. Maas, A. Magyar, H. Mao, M. Moreto, A. Ou, D. A. Patterson, B. Richards, C. Schmidt, S. Twigg, H. Vo, and A. Waterman, "The Rocket Chip Generator," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17, Apr 2016.

[7] G. Baccelli, D. Stathis, A. Hemani, and M. Martina, "NACU: A Non-Linear Arithmetic Unit for Neural Networks," in *2020 57th ACM/IEEE Design Automation Conference (DAC)*, 2020, pp. 1–6.

[8] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avižienis, J. Wawrzynek, and K. Asanović, "Chisel: Constructing hardware in a Scala embedded language," in *DAC Design Automation Conference 2012*, 2012, pp. 1212–1221.

[9] R. P. Brent, "An algorithm with guaranteed convergence for finding a zero of a function," *The Computer Journal*, vol. 14, no. 4, pp. 422–425, 01 1971. [Online]. Available: https://doi.org/10.1093/comjnl/14.4.422

[10] H. Chen, Z. Yu, J. Xu, L. Jiang, Z. Lu, Y. Fu, and L. Li, "Huicore: A Generalized Hardware Accelerator for Complicated Functions," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 69, no. 6, pp. 2463–2476, 2022.

[11] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 367–379.

[12] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, "Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge," 2018. [Online]. Available: https://arxiv.org/abs/1803.05457

[13] H. Cruz, P. Flores, M. Vestias, J. Monteiro, H. Neto, and R. P. Duarte, "Algorithm-specific Optimizations for On-Board Real-Time Backprojection on FPGA," in *EUSAR 2024; 15th European Conference on Synthetic Aperture Radar*, 2024, pp. 54–59.

[14] V. Dadu, J. Weng, S. Liu, and T. Nowatzki, "Towards General Purpose Acceleration by Exploiting Common Data-Dependence Forms," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '52. New York, NY, USA: Association for Computing Machinery, 2019, p. 924–939.

[15] Y. Dai, X. Gao, H. Lin, W. Yin, W.-S. Luk, and L. Wang, "Dependency-Aware Data Parallelism on Spatial CGRA via Constraint Satisfaction and Graph Coloring," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, pp. 1–1, 2025.

[16] Y. Dai, X. Gao, Y. Qiu, J. Li, Y. Cao, Y. Mao, S. Chen, W. Yin, W.-S. Luk, and L. Wang, "Coffa: A co-design framework for fused-grained reconfigurable architecture towards efficient irregular loop handling," *IEEE Transactions on Computers*, vol. 74, no. 9, pp. 3099–3113, 2025.

[17] Y. Dai, X. Gao, C. Shen, B. Peng, W. Yin, W.-S. Luk, and L. Wang, "Towards Efficient Data Parallelism on Spatial CGRA via Constraint Satisfaction and Graph Coloring," in *Proceedings of the 30th Asia and South Pacific Design Automation Conference*, ser. ASPDAC '25, 2025, p. 1023–1030.

[18] E. Darulova and A. Volkova, "Sound Approximation of Programs with Elementary Functions," in *Computer Aided Verification*, I. Dillig and S. Tasiran, Eds. Cham: Springer International Publishing, 2019, pp. 174–183.

[19] F. de Dinechin and B. Pasca, "Designing Custom Arithmetic Data Paths with FloPoCo," *IEEE Design & Test of Computers*, vol. 28, no. 4, pp. 18–27, 2011.

[20] DeepSeek-AI, :, X. Bi, D. Chen, G. Chen, S. Chen, D. Dai, C. Deng, H. Ding, K. Dong, Q. Du, Z. Fu, H. Gao, K. Gao, W. Gao, R. Ge, K. Guan, D. Guo, J. Guo, G. Hao, Z. Hao, Y. He, W. Hu, P. Huang, E. Li, G. Li, J. Li, Y. Li, Y. K. Li, W. Liang, F. Lin, A. X. Liu, B. Liu, W. Liu, X. Liu, X. Liu, Y. Liu, H. Lu, S. Lu, F. Luo, S. Ma, X. Nie, T. Pei, Y. Piao, J. Qiu, H. Qu, T. Ren, Z. Ren, C. Ruan, Z. Sha, Z. Shao, J. Song, X. Su, J. Sun, Y. Sun, M. Tang, B. Wang, P. Wang, S. Wang, Y. Wang, Y. Wang, T. Wu, Y. Wu, X. Xie, Z. Xie, Z. Xie, Y. Xiong, H. Xu, R. X. Xu, Y. Xu, D. Yang, Y. You, S. Yu, X. Yu, B. Zhang, H. Zhang, L. Zhang, L. Zhang, M. Zhang, M. Zhang, W. Zhang, Y. Zhang, C. Zhao, Y. Zhao, S. Zhou, S. Zhou, Q. Zhu, and Y. Zou, "DeepSeek LLM: Scaling Open-Source Language Models with Longtermism," 2024. [Online]. Available: https://arxiv.org/abs/2401.02954

[21] E. Del Sozzo, X. Wang, B. Adhi, C. Cortes, J. Anderson, and K. Sano, "Exploration of Trade-offs Between General-Purpose and Specialized Processing Elements in HPC-Oriented CGRA," in *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2024, pp. 668–680.

[22] J. Deng, X. Tang, J. Zhang, Y. Li, L. Zhang, B. Han, H. He, F. Tu, L. Liu, S. Wei, Y. Hu, and S. Yin, "Towards Efficient Control Flow Handling in Spatial Architecture via Architecting the Control Flow Plane," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 1395–1408.

[23] H. Du, C. Wen, Z. Chen, L. Zhang, Q. Sun, Z. Yan, and C. Zhuo, "Algorithm-Hardware Co-Design of a Unified Accelerator for Non-Linear Functions in Transformers," in *2025 Design, Automation & Test in Europe Conference (DATE)*, 2025, pp. 1–7.

[24] Efficient Computer., https://www.efficient.computer.

[25] X. Feng, Y. Li, Y. Qian, J. Gao, W. Cao, and L. Wang, "A High-Precision Flexible Symmetry-Aware Architecture for Element-Wise Activation Functions," in *2021 International Conference on Field-Programmable Technology (ICFPT)*, 2021, pp. 1–4.

[26] G. Gobieski, A. O. Atli, K. Mai, B. Lucia, and N. Beckmann, "Snafu: An Ultra-Low-Power, Energy-Minimal CGRA-Generation Framework and Architecture," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 1027–1040.

[27] G. Gobieski, S. Ghosh, M. Heule, T. Mowry, T. Nowatzki, N. Beckmann, and B. Lucia, "RipTide: A Programmable, Energy-Minimal Dataflow Compiler and Architecture," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 546–564.

[28] T. J. Ham, L. Wu, N. Sundaram, N. Satish, and M. Martonosi, "Graphicionado: A high-performance and energy-efficient accelerator for graph analytics," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016, pp. 1–13.

[29] K. Han, J. Ahn, and K. Choi, "Power-Efficient Predication Techniques for Acceleration of Control Flow Execution on CGRA," *ACM Trans. Archit. Code Optim.*, vol. 10, no. 2, may 2013.

[30] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, "EIE: Efficient Inference Engine on Compressed Deep Neural Network," in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 243–254.

[31] J. F. Hart, *Computer Approximations*. USA: Krieger Publishing Co., Inc., 1978.

[32] Q. Hong, Z. Liu, Q. Long, H. Tong, T. Zhang, X. Zhu, Y. Zhao, H. Ru, Y. Zha, Z. Zhou, J. Wu, H. Tan, W. Hong, Y. Xu, and X. Guo, "A reconfigurable multi-precision quantization-aware nonlinear activation function hardware module for DNNs," *Microelectronics Journal*, vol. 151, p. 106346, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S187923912400050X

[33] W. G. Horner, "XXI. A new method of solving numerical equations of all orders, by continuous approximation," *Philosophical Transactions of the Royal Society of London*, pp. 308 – 335, 1819. [Online]. Available: https://api.semanticscholar.org/CorpusID:186210512

[34] A. Howard, M. Sandler, B. Chen, W. Wang, L.-C. Chen, M. Tan, G. Chu, V. Vasudevan, Y. Zhu, R. Pang, H. Adam, and Q. Le, "Searching for MobileNetV3," in *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 1314–1324.

[35] S.-F. Hsiao, C.-S. Wen, Y.-H. Chen, and K.-C. Huang, "Hierarchical

Multipartite Function Evaluation," *IEEE Transactions on Computers*, vol. 66, no. 1, pp. 89–99, 2017.

[36] J. Hu, L. Shen, and G. Sun, "Squeeze-and-Excitation Networks," in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 7132–7141.

[37] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mistral 7B," 2023. [Online]. Available: https://arxiv.org/abs/2310.06825

[38] M. Karunaratne, A. K. Mohite, T. Mitra, and L.-S. Peh, "HyCUBE: A CGRA with reconfigurable single-cycle multi-hop interconnect," in *2017 54th ACM/EDAC/IEEE Design Automation Conference (DAC)*, 2017, pp. 1–6.

[39] S. Y. Kim, C. H. Kim, W. J. Lee, I. Park, and S. W. Kim, "Low-overhead inverted LUT design for bounded DNN activation functions on floating-point vector ALUs," *Microprocess. Microsyst.*, vol. 93, no. C, Sep. 2022. [Online]. Available: https://doi.org/10.1016/j.micpro.2022.104592

[40] C. G. Lee, "UTDSP Benchmark Suite," https://www.eecg.toronto.edu/~corinna/DSP/infrastructure/.

[41] C.-W. Liu, S.-H. Ou, K.-C. Chang, T.-C. Lin, and S.-K. Chen, "A Low-Error, Cost-Efficient Design Procedure for Evaluating Logarithms to Be Used in a Logarithmic Arithmetic Processor," *IEEE Transactions on Computers*, vol. 65, no. 4, pp. 1158–1164, 2016.

[42] S. Liu, J. Weng, D. Kupsh, A. Sohrabizadeh, Z. Wang, L. Guo, J. Liu, M. Zhulin, R. Mani, L. Zhang, J. Cong, and T. Nowatzki, "OverGen: Improving FPGA Usability through Domain-specific Overlay Generation," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 35–56.

[43] M. Loukrakpam and M. Choudhury, "Error-Aware Design Procedure to Implement Hardware-Efficient Logarithmic Circuits," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 67, no. 5, pp. 851–855, 2020.

[44] T.-K. Luong, V.-T. Nguyen, A.-T. Nguyen, and E. Popovici, "Efficient Architectures and Implementation of Arithmetic Functions Approximation Based Stochastic Computing," in *2019 IEEE 30th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, vol. 2160-052X, 2019, pp. 281–287.

[45] Y. Mao, X. Gao, J. Lou, Y. Qiu, W. Yin, W.-S. Luk, and L. Wang, "CFE-ACT: A CGRA-based Framework Enabling Agile CNN and Transformer Accelerator Design," in *2024 34th International Conference on Field-Programmable Logic and Applications (FPL)*, 2024, pp. 213–219.

[46] J. N. Mitchell, "Computer Multiplication and Division Using Binary Logarithms," *IRE Transactions on Electronic Computers*, vol. EC-11, no. 4, pp. 512–517, 1962.

[47] S. M. Mohamed, W. S. Sayed, A. G. Radwan, and L. A. Said, "FPGA Implementation of Reconfigurable CORDIC Algorithm and a Memristive Chaotic System With Transcendental Nonlinearities," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 69, no. 7, pp. 2885–2892, 2022.

[48] B.-G. Nam, H. Kim, and H.-J. Yoo, "A Low-Power Unified Arithmetic Unit for Programmable Handheld 3-D Graphics Systems," *IEEE Journal of Solid-State Circuits*, vol. 42, no. 8, pp. 1767–1778, 2007.

[49] A. Niktash, H. T. Parizi, A. H. Kamalizad, and N. Bagherzadeh, "Recfec: A reconfigurable fec processor for viterbi, turbo, reed-solomon and ldpc coding," in *2008 IEEE Wireless Communications and Networking Conference*, 2008, pp. 605–610.

[50] P. Nilsson, A. U. R. Shaik, R. Gangarajaiah, and E. Hertz, "Hardware implementation of the exponential function using Taylor series," in *2014 NORCHIP*, 2014, pp. 1–4.

[51] R. Prabhakar, Y. Zhang, D. Koeplinger, M. Feldman, T. Zhao, S. Hadjis, A. Pedram, C. Kozyrakis, and K. Olukotun, "Plasticine: A reconfigurable architecture for parallel patterns," in *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 389–402.

[52] A. S. Prasad, G. İslamoğlu, L. Bertaccini, D. Rossi, F. Conti, and L. Benini, "Pace: An optimal piecewise polynomial approximation unit for flexible and efficient transformer non-linearity acceleration," in *2025 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, vol. 1, 2025, pp. 1–6.

[53] ——, "PACE-Lite: Compact and Efficient Piecewise Polynomial Approximation for Transformer Nonlinearity Acceleration," in *2025 IEEE 43rd International Conference on Computer Design (ICCD)*, 2025, pp. 111–118.

[54] R. Prasad, "Nx-cgra: A programmable hardware accelerator for core transformer algorithms on edge devices," 2025. [Online]. Available: https://arxiv.org/abs/2511.17235

[55] J. Qin, "CGRA-Nonlinear-Benchmark," https://github.com/HobbitQia/cgra-nonlinear-benchmark.

[56] J. Qin, T. Xia, C. Tan, J. Zhang, and S. Q. Zhang, "PICACHU: Plug-In CGRA Handling Upcoming Nonlinear Operations in LLMs," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 845–861. [Online]. Available: https://doi.org/10.1145/3676641.3716013

[57] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," 2019. [Online]. Available: https://api.semanticscholar.org/CorpusID:160025533

[58] M. Roemmele, C. Bejan, and A. Gordon, "Choice of Plausible Alternatives: An Evaluation of Commonsense Causal Reasoning." in *AAAI Spring Symposium - Technical Report*, 01 2011.

[59] SambaNova System., https://sambanova.ai.

[60] T. Santos, "HLS Projects," https://github.com/tiagolascasas/HLS-Projects.

[61] N. Serafin, S. Ghosh, H. Desai, N. Beckmann, and B. Lucia, "Pipestitch: An energy-minimal dataflow architecture with lightweight threads," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 1409–1422. [Online]. Available: https://doi.org/10.1145/3613424.3614283

[62] K. Shi, X. Zhou, H. Zhou, and L. Wang, "An optimized gib routing architecture with bent wires for fpga," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 16, no. 1, dec 2022.

[63] A. Talmor, J. Herzig, N. Lourie, and J. Berant, "CommonsenseQA: A question answering challenge targeting commonsense knowledge," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4149–4158. [Online]. Available: https://aclanthology.org/N19-1421/

[64] C. Tan, M. Jiang, D. Patil, Y. Ou, Z. Li, L. Ju, T. Mitra, H. Park, A. Tumeo, and J. Zhang, "ICED: An Integrated CGRA Framework Enabling DVFS-Aware Acceleration," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024, pp. 1338–1352.

[65] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 09–15 Jun 2019, pp. 6105–6114. [Online]. Available: https://proceedings.mlr.press/v97/tan19a.html

[66] C. Torng, P. Pan, Y. Ou, C. Tan, and C. Batten, "Ultra-Elastic CGRAs for Irregular Loop Specialization," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 412–425.

[67] A. Vasilyev, N. Bhagdikar, A. Pedram, S. Richardson, S. Kvatinsky, and M. Horowitz, "Evaluating programmable architectures for imaging and vision applications," in *The 49th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO-49. IEEE Press, 2016.

[68] M. Vershik, A. N. Malozemov, V. and B. Pevnyi, A. "Best Piecewise Polynomial Approximation," *Siberian Mathematical Journal*, vol. 16, no. 5, pp. 706–717, 1975.

[69] L. S. N. Vulchi, P. Valipireddy, M. Basavaraju, and M. Rao, *HyPPO: Hybrid Piece-wise Polynomial Approximation and Optimization for Hardware Efficient Designs*. New York, NY, USA: Association for Computing Machinery, 2025, p. 230–236. [Online]. Available: https://doi.org/10.1145/3658617.3697713

[70] B. Wang, M. Karunarathne, A. Kulkarni, T. Mitra, and L.-S. Peh, "HyCUBE: A 0.9V 26.4 MOPS/mW, 290 pJ/op, Power Efficient Accelerator for IoT Applications," in *2019 IEEE Asian Solid-State Circuits Conference (A-SSCC)*, 2019, pp. 133–136.

[71] J. Weng, S. Liu, V. Dadu, Z. Wang, P. Shah, and T. Nowatzki, "DSAGEN: Synthesizing Programmable Spatial Accelerators," in *ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 268–281.

[72] J. Weng, S. Liu, D. Kupsh, and T. Nowatzki, "Unifying Spatial Accelerator Compilation With Idiomatic and Modular Transformations," *IEEE Micro*, vol. 42, no. 5, pp. 59–69, 2022.
[73] J. Weng, S. Liu, Z. Wang, V. Dadu, and T. Nowatzki, "A Hybrid Systolic-Dataflow Architecture for Inductive Matrix Algorithms," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2020, pp. 703–716.
[74] D. Wijerathne, Z. Li, M. Karunarathne, A. Pathania, and T. Mitra, "CASCADE: High Throughput Data Streaming via Decoupled Access-Execute CGRA," *ACM Trans. Embed. Comput. Syst.*, vol. 18, no. 5s, oct 2019.
[75] S. J. E. Wilton, "Architectures and Algorithms for Field-Programmable Gate Arrays with Embedded Memory," *Ph.D. Dissertation, University of Toronto*, 1997.
[76] X. Wu, S. Liang, M. Wang, and Z. Wang, "ReAFM: A Reconfigurable Nonlinear Activation Function Module for Neural Networks," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 70, no. 7, pp. 2660–2664, 2023.
[77] Y. Xie, A. N. Joseph Raj, Z. Hu, S. Huang, Z. Fan, and M. Joler, "A Twofold Lookup Table Architecture for Efficient Approximation of Activation Functions," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 12, pp. 2540–2550, 2020.
[78] S. Yoo, H. Kim, J. Kim, S. Park, J.-Y. Kim, and J. Oh, "LightTrader: A Standalone High-Frequency Trading System with Deep Learning Inference Accelerators and Proactive Scheduler," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023, pp. 1017–1030.
[79] Z. Yuan, S. Yuan, P. Liu, C. Yin, L. Xu, W. Sheng, and N. Jing, "A Flexible and High-Precision Activation Function Unit Based on Equi-Error Partitioning Algorithm," in *2024 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2024, pp. 1–5.
[80] B. Zamanlooy and M. Mirhassani, "Efficient VLSI Implementation of Neural Networks With Hyperbolic Tangent Activation Function," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 22, no. 1, pp. 39–48, 2014.
[81] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, "HellaSwag: Can a Machine Really Finish Your Sentence?" 2019. [Online]. Available: https://arxiv.org/abs/1905.07830
[82] G. Zou, Y. Dai, W. Yin, and L. Wang, "Towards Efficient Logarithmic Converter Circuit Design via Constraint-driven Parameter Exploration," *IEEE Transactions on Circuits and Systems II: Express Briefs*, pp. 1–5, 2025.

ARTIFACT APPENDIX

A. Abstract

The artifact contains the artifact evaluation for this work. First, we provide the checklist for this artifact. Next, we describe the directory structure for the code. Finally, we illustrate how to use the artifact to reproduce results and extend the implementation. Since the experiments involve multiple platforms, it is difficult to run all of them within a single Docker. Therefore, while all source codes are provided, the provided Docker is intended to reproduce part of the results, which highlight the advantages of *LoRA* over state-of-the-art approaches.

B. Artifact check-list (meta-information)

- **Algorithm:** Chebyshev-based Approximation
- **Data set:** https://github.com/Dai-dirk/COFFA/tree/LoRA-ISCA-AE/System_level_benchmarks
- **Run-time environment:** Ubuntu 20.04 or an Ubuntu Docker image with root access.
- **Hardware:** Any machine that can run a Docker environment. For the evaluations on Section VIII-B-3, different hardware is required, such as an NVIDIA 4090 GPU or an NVIDIA A100 GPU.
- **Metrics:** MAE, MSE, end-to-end accuracy, number of cycles for SoC execution.
- **Output:** The approximation result for the non-linear function, the end-to-end accuracy for three applications (i.e., DCT, DNN, LLM), and the number of cycles for SoC execution.
- **Experiments:** See the README or Manual within the open-source repository for details.
- **How much disk space is required (approximately)?:** 23GB.
- **How much time is needed to prepare workflow (approximately)?:** 2 days (depends on the Internet speed and whether the Chipyard can be installed successfully).
- **How much time is needed to complete experiments (approximately)?:** 14 days (including the proposed three-level evaluations).
- **Publicly available?:** Yes
- **Code licenses (if publicly available)?:** Open Source Initiative BSD 3-Clause License.
- **Archived (provide DOI)?:** https://doi.org/10.5281/zenodo.19447155

C. Description

1) How to access: The source code for the LoRA framework can be accessed at: https://github.com/Dai-dirk/COFFA/tree/LoRA-ISCA-AE. The Docker image is at: docker.cnb.cool/fudaneda/docker/chipyard. The Zenodo DOI URL is at: https://doi.org/10.5281/zenodo.19447155.

D. Installation

To facilitate the evaluation of LoRA, we strongly recommend using the provided Docker environment. Alternatively, users can follow the instructions in the provided README file to perform their own experiments.

E. Suggested Experiment workflow

Because the experiments are numerous and span multiple hardware platforms, to facilitate artifact evaluation, we introduce how to execute some of the experiments from the three-level evaluation in this paper within the provided Docker environment.

First, pull the Docker to the local environment:

```
docker run -it docker.cnb.cool/fudaneda/docker/chipyard /bin/bash
```

Then, locate the work directory and set up the path:

```
cd /root
source ./setup_path.sh
```

1) Algorithm-level evaluation:

- Locate the script:
  `cd /root/PiecewiseChebFitter`
- Run the sript:
  `python3 run_func.py <function name>`

The `function name` include: gelu, sigmoid, softplus, swish, and tanh. The configuration file for each function can be found in the *config* folder.

2) Unit-level evaluation:

- Locate the script:
  `cd /root/DCT`
- To run DCT by the accurate baseline:
  - Run the script:
    `python3 test_set12.py --quant-scale <0.1/0.75/1.5>`

- To run DCT by PICACHU:
  - Run the script:
    ```
    python3 test_set12_tylor.py
    --taylor-order <4/6/8>
    --quant-scale <0.1/0.75/1.5>
    ```

  It's worth noting that the *cos* function is an even function, hence `--taylor-order <4/6/8>` corresponds to 3/4/5 polynomial terms in TABLE VIII.
- To run DCT by *LoRA*-A:
  - Run the script:
    ```
    python3 test_set12_lns_float_XCoreA.py
    --quant-scale <0.1/0.75/1.5>
    ```
- To run DCT by *LoRA*-B:
  - Run the script:
    ```
    python3 test_set12_lns_float_XCoreB.py
    --quant-scale <0.1/0.75/1.5>
    ```
- To run DCT by *LoRA*-C:
  - Run the script:
    ```
    python3 test_set12_lns_float_XCoreC.py
    --quant-scale <0.1/0.75/1.5>
    ```

3) System-level evaluation:

- Change to the chipyard directory:
  `cd /root/chipyard`
- Activate the conda environment:
  `conda activate ./.conda-env`
- Locate the script:
  `cd /root/chipyard/generators/fgra`

Due to the EDA license, we cannot provide access to Synopsys VCS. As a solution, we provide the Verilator-based simulator for the evaluation.

- To run the simulation of *LoRA*:
  - Build the simulator:
    `python3 build_verilator.py LoRA`
  - Run the simulation:
    `python3 run_verilator_lora.py`
  - Run the simulation with loop unrolling:
    `python3 run_verilator_lora_unroll.py`
- To run the simulation of PICACHU:
  - Build the simulator:
    `python3 build_verilator.py PICACHU`
  - Run the simulation:
    `python3 run_verilator_picachu.py`
  - Run the simulation with loop unrolling:
    `python3 run_verilator_picachu_unroll.py`

F. Expected results

1) Algorithm-level evaluation: The expected results in this level are the approximation results generated by the software, which can be found in the folders *fig* and *result*.

2) Unit-level evaluation: The expected results in this level are related to TABLE VIII.

3) System-level evaluation: The expected results after running the simulation are related to Fig. 10, where the simulator will report the number of cycles for each benchmark.

In addition, we provide the area reports for the SoCs in Fig. 9, the reports can be found at https://github.com/Dai-dirk/COFFA/tree/LoRA-ISCA-AE/SoC_Area_Report.

G. Methodology

Submission, reviewing, and badging methodology:

- https://www.acm.org/publications/policies/artifact-review-and-badging-current
- https://cTuning.org/ae